Move from Windows Runtime 8.x to UWP

Article • 10/20/2022

If you have a Universal 8.1 app—whether it's targeting Windows 8.1, Windows Phone 8.1, or both—then you'll find that your source code and skills will port smoothly to Windows 10. With Windows 10, you can create a Universal Windows Platform (UWP) app, which is a single app package that your customers can install onto every kind of device. For more background on Windows 10, UWP apps, and the concepts of adaptive code and adaptive UI that we'll mention in this porting guide, see Guide to UWP apps.

While porting, you'll find that Windows 10 shares the majority of APIs with the previous platforms, as well as XAML markup, UI framework, and tooling, and you'll find it all reassuringly familiar. Just as before, you can still choose between C++, C#, and Visual Basic for the programming language to use along with the XAML UI framework. Your first steps in planning exactly what to do with your current app or apps will depend on the kinds of apps and projects you have. That's explained in the following sections.

If you have a Universal 8.1 app

A Universal 8.1 app is built from an 8.1 Universal App project. Let's say the project's name is AppName_81. It contains these sub-projects.

- AppName_81.Windows. This is the project that builds the app package for Windows 8.1.
- AppName_81.WindowsPhone. This is the project that builds the app package for Windows Phone 8.1.
- AppName_81.Shared. This is the project that contains source code, markup files, and other assets and resources that are used by both of the other two projects.

Often, an 8.1 Universal Windows app offers the same features—and does so using the same code and markup—in both its Windows 8.1 and Windows Phone 8.1 forms. An app like that is an ideal candidate for porting to a single Windows 10 app that targets the Universal device family (and that you can install onto the widest range of devices). You'll essentially port the contents of the Shared project and you'll need to use little or nothing from the other two projects because there'll be little or nothing in them.

Other times, the Windows 8.1 and/or the Windows Phone 8.1 form of the app contain unique features. Or they contain the same features but they implement those features using different techniques or different technology. With an app like that, you can choose

to port it to a single app that targets the Universal device family (in which case you will want the app to adapt itself to different devices), or you can choose to port it as more than one app, perhaps one targeting the Desktop device family and another targeting the Mobile device family. The nature of the Universal 8.1 app will determine which of these options is best for your case.

1. Port the contents of the Shared project to an app targeting the Universal device family. If applicable, salvage any other content from the Windows and WindowsPhone projects, and use that content either unconditionally in the app or conditional on the device that your app happens to be running on at the time (the latter behavior is known as *adaptive*).
2. Port the contents of the WindowsPhone project to an app targeting the Universal device family. If applicable, salvage any other content from the Windows project, using it either unconditionally or adaptively.
3. Port the contents of the Windows project to an app targeting the Universal device family. If applicable, salvage any other content from the WindowsPhone project, using it either unconditionally or adaptively.
4. Port the contents of the Windows project to an app targeting the Universal or the Desktop device family and also port the contents of the WindowsPhone project to an app targeting the Universal or the Mobile device family. You can create a solution with a Shared project, and continue to share source code, markup files, and other assets and resources between the two projects. Or, you can create different solutions and still share the same items using links.

If you have a Windows 8.1 app

Port the project to an app targeting the Universal or the Desktop device family. If you choose the Universal device family, and your app calls APIs that are implemented only in the Desktop device family, then you can guard those calls with adaptive code.

If you have a Windows Phone 8.1 app

Port the project to an app targeting the Universal or the Mobile device family. If you choose the Universal device family, and your app calls APIs that are implemented only in the Mobile device family, then you can guard those calls with adaptive code.

Adapting your app to multiple form factors

The option you choose from the previous sections will determine the range of devices that your app or apps will run on, and that may well be a very wide range of devices.

Even limiting your app to the Mobile device family still leaves you with a wide range of screen sizes to support. So, if your app will be running on form factors that it didn't formerly support, then test your UI on those form factors and make any change necessary, so that your UI adapts appropriately on each. You can think of this is a post-porting task, or a porting stretch-goal, and there are some examples of it in practice in the Bookstore2 and QuizGame case studies.

Approaching porting layer-by-layer

When porting a Universal 8.1 app to the model for UWP apps, virtually all of your knowledge and experience will transfer, as will most of your source code and markup and the software patterns you use.

- **View**. The view (together with the view model) makes up your app's UI. Ideally, the view consists of markup bound to observable properties of a view model. Another pattern (common and convenient, but only in the short term) is for imperative code in a code-behind file to directly manipulate UI elements. In either case, your UI markup and design—and even imperative code that manipulates UI elements—will be straightforward to port.
- **View models and data models**. Even if you don't formally embrace separation-of-concerns patterns (such as MVVM), there is inevitably code present in your app that performs the function of view model and data model. View model code makes use of types in the UI framework namespaces. Both view model and data model code also use non-visual operating system and .NET Framework APIs (including APIs for data-access). And those APIs are available for UWP apps, too, so most if not all of this code will port without change.
- **Cloud services**. It's likely that some of your app (perhaps a great deal of it) runs in the cloud in the form of services. The part of the app running on the client device connects to those. This is the part of a distributed app most likely to remain unchanged when porting the client part. If you don't already have one, a good cloud services option for your UWP app is Microsoft Azure Mobile Services, which provides powerful back-end components that your app can call for services ranging from simple notifications for live tiles updates up to the kind of heavy-lifting scalability a server farm can provide.

Before or during the porting, consider whether your app could be improved by refactoring it so that code with a similar purpose is gathered together in layers and not scattered arbitrarily. Factoring your app into layers like those described above makes it easier for you to make your app correct, to test it, and then subsequently to read and maintain it. You can make functionality more reusable by following the Model-View-ViewModel (MVVM) pattern. This pattern keeps the data, business, and UI parts of your

app separate from one another. Even within the UI it can keep state and behavior separate, and separately testable, from the visuals. With MVVM, you can write your data and business logic once and use it on all devices no matter the UI. It's likely that you'll be able to re-use much of the view model and view parts across devices, too.

Topic	Description
Porting the project	You have two options when you begin the porting process. One is to edit a copy of your existing project files, including the app package manifest (for that option, see the info about updating your project files in Migrate apps to the Universal Windows Platform (UWP)). The other option is to create a new Windows 10/11 project in Visual Studio and copy your files into it.
Troubleshooting	We highly recommend reading to the end of this porting guide, but we also understand that you're eager to forge ahead and get to the stage where your project builds and runs. To that end, you can make temporary progress by commenting or stubbing out any non-essential code, and then returning to pay off that debt later. The table of troubleshooting symptoms and remedies in this topic may be helpful to you at this stage, although it's not a substitute for reading the next few topics. You can always refer back to the table as you progress through the later topics.
Porting XAML and UI	The practice of defining UI in the form of declarative XAML markup translates extremely well from Universal 8.1 apps to UWP apps. You'll find that most of your markup is compatible, although you may need to make some adjustments to the system Resource keys or custom templates that you're using.
Porting for I/O, device, and app model	Code that integrates with the device itself and its sensors involves input from, and output to, the user. It can also involve processing data. But, this code is not generally thought of as either the UI layer or the data layer. This code includes integration with the vibration controller, accelerometer, gyroscope, microphone and speaker (which intersect with speech recognition and synthesis), (geo)location, and input modalities such as touch, mouse, keyboard, and pen.
Case study: Bookstore1	This topic presents a case study of porting a very simple Universal 8.1 app to a Windows 10 and Windows 11 UWP app. A Universal 8.1 app is one that builds one app package for Windows 8.1, and a different app package for Windows Phone 8.1. With Windows 10 and Windows 11, you can create a single app package that your customers can install onto a wide range of devices, and that's what we'll do in this case study. See Guide to UWP apps.
Case study: Bookstore2	This case study—which builds on the info given in SemanticZoom control. In the view model, each instance of the class Author represents the group of the books written by that author, and in the SemanticZoom, we can either view the list of books grouped by author or we can zoom out to see a jump list of authors.

Topic	Description
Case study: QuizGame	This topic presents a case study of porting a functioning peer-to-peer quiz game WinRT 8.1 sample app to a Windows 10 and Windows 11 UWP app.

Related topics

Documentation

- Windows Runtime reference
- Building Universal Windows apps for all Windows devices
- Designing UX for apps

Porting a Windows Runtime 8.x project to a UWP project

Article • 03/16/2023

You have two options when you begin the porting process. One is to edit a copy of your existing project files, including the app package manifest (for that option, see the info about updating your project files in Migrate apps to the Universal Windows Platform (UWP)). The other option is to create a new Windows 10 project in Visual Studio and copy your files into it. The first section in this topic describes that second option, but the rest of the topic has additional info applicable to both options. You can also choose to keep your new Windows 10 project in the same solution as your existing projects and share source code files using a shared project. Or, you can keep the new project in a solution of its own and share source code files using the linked files feature in Visual Studio.

Create the project and copy files to it

These steps focus on the option to create a new Windows 10 project in Visual Studio and copy your files into it. Some of the specifics around how many projects you create, and which files you copy over, will depend on the factors and decisions described in If you have a Universal 8.1 app and the sections that follow it. These steps assume the simplest case.

1. Launch Microsoft Visual Studio 2015 and create a new Blank Application (Windows Universal) project. For more info, see Jumpstart your Windows Runtime 8.x app using templates (C#, C++, Visual Basic). Your new project builds an app package (an appx file) that will run on all device families.
2. In your Universal 8.1 app project, identify all the source code files and visual asset files that you want to reuse. Using File Explorer, copy data models, view models, visual assets, Resource Dictionaries, folder structure, and anything else that you wish to re-use, to your new project. Copy or create sub-folders on disk as necessary.
3. Copy views (for example, MainPage.xaml and MainPage.xaml.cs) into the new project, too. Again, create new sub-folders as necessary, and remove the existing views from the project. But, before you over-write or remove a view that Visual Studio generated, keep a copy because it may be useful to refer to it later. The first phase of porting a Universal 8.1 app focuses on getting it to look good and work well on one device family. Later, you'll turn your attention to making sure the views

adapt themselves well to all form factors, and optionally to adding any adaptive code to get the most from a particular device family.

4. In **Solution Explorer**, make sure **Show All Files** is toggled on. Select the files that you copied, right-click them, and click **Include In Project**. This will automatically include their containing folders. You can then toggle **Show All Files** off if you like. An alternative workflow, if you prefer, is to use the **Add Existing Item** command, having created any necessary sub-folders in the Visual Studio **Solution Explorer**. Double-check that your visual assets have **Build Action** set to **Content** and **Copy to Output Directory** set to **Do not copy**.
5. You are likely to see some build errors at this stage. But, if you know what you need to change, then you can use Visual Studio's **Find and Replace** command to make bulk changes to your source code; and in the imperative code editor in Visual Studio, use the **Resolve** and **Organize Usings** commands on the context menu for more targeted changes.

Maximizing markup and code reuse

You will find that refactoring a little, and/or adding adaptive code (which is explained below), will allow you to maximize the markup and code that works across all device families. Here are more details.

- Files that are common to all device families need no special consideration. Those files will be used by the app on all the device families that it runs on. This includes XAML markup files, imperative source code files, and asset files.
- It is possible for your app to detect the device family that it is running on and navigate to a view that has been designed specifically for that device family. For more details, see Detecting the platform your app is running on.
- A similar technique that you may find useful if there is no alternative is give a markup file or **ResourceDictionary** file (or the folder that contains the file) a special name such that it is automatically loaded at runtime only when your app runs on a particular device family. This technique is illustrated in the Bookstore1 case study.
- You should be able to remove a lot of the conditional compilation directives in your Universal 8.1 app's source code if you only need to support Windows 10. See Conditional compilation and adaptive code in this topic.
- To use features that are not available on all device families (for example, printers, scanners, or the camera button), you can write adaptive code. See the third example in Conditional compilation and adaptive code in this topic.
- If you want to support Windows 8.1, Windows Phone 8.1, and Windows 10, then you can keep three projects in the same solution and share code with a Shared project. Alternatively, you can share source code files between projects. Here's how: in Visual Studio, right-click the project in **Solution Explorer**, select **Add**

Existing Item, select the files to share, and then click **Add As Link**. Store your source code files in a common folder on the file system where the projects that link to them can see them. And don't forget to add them to source control.

- For reuse at the binary level, rather than the source code level, see Creating Windows Runtime components in C# and Visual Basic. There are also Portable Class Libraries, which support the subset of .NET APIs that are available in the .NET Framework for Windows 8.1, Windows Phone 8.1, and Windows 10 apps (.NET Core), and the full .NET Framework. Portable Class Library assemblies are binary compatible with all these platforms. Use Visual Studio to create a project that targets a Portable Class Library. See Cross-Platform Development with the Portable Class Library.

Extension SDKs

Most of the Windows Runtime APIs your Universal 8.1 app already calls are implemented in the set of APIs known as the universal device family. But, some are implemented in extension SDKs, and Visual Studio only recognizes APIs that are implemented by your app's target device family or by any extension SDKs that you have referenced.

If you get compile errors about namespaces or types or members that could not be found, then this is likely to be the cause. Open the API's topic in the API reference documentation and navigate to the Requirements section: that will tell you what the implementing device family is. If that's not your target device family, then to make the API available to your project, you will need a reference to the extension SDK for that device family.

Click **Project** > **Add Reference** > **Windows Universal** > **Extensions** and select the appropriate extension SDK. For example, if the APIs you want to call are available only in the mobile device family, and they were introduced in version 10.0.x.y, then select **Windows Mobile Extensions for the UWP**.

That will add the following reference to your project file:

XML

```
<ItemGroup>
    <SDKReference Include="WindowsMobile, Version=10.0.x.y">
        <Name>Windows Mobile Extensions for the UWP</Name>
    </SDKReference>
</ItemGroup>
```

The name and version number match the folders in the installed location of your SDK. For example, the above information matches this folder name:

```
\Program Files (x86)\Windows Kits\10\Extension SDKs\WindowsMobile\10.0.x.y
```

Unless your app targets the device family that implements the API, you'll need to use the **ApiInformation** class to test for the presence of the API before you call it (this is called adaptive code). This condition will then be evaluated wherever your app runs, but it will only evaluate to true on devices where the API is present and therefore available to call. Only use extension SDKs and adaptive code after first checking whether a universal API exists. Some examples are given in the section below.

Also, see App package manifest.

Conditional compilation and adaptive code

If you're using conditional compilation (with C# preprocessor directives) so that your code files work on both Windows 8.1 and Windows Phone 8.1, then you can now review that conditional compilation in light of the convergence work done in Windows 10. Convergence means that, in your Windows 10 app, some conditions can be removed altogether. Others change to run-time checks, as demonstrated in the examples below.

Note If you want to support Windows 8.1, Windows Phone 8.1, and Windows 10 in a single code file, then you can do that too. If you look in your Windows 10 project at the project properties pages, you'll see that the project defines WINDOWS_UAP as a conditional compilation symbol. So, you can use that in combination with WINDOWS_APP and WINDOWS_PHONE_APP. These examples show the simpler case of removing the conditional compilation from a Universal 8.1 app and substituting the equivalent code for a Windows 10 app.

This first example shows the usage pattern for the **PickSingleFileAsync** API (which applies only to Windows 8.1) and the **PickSingleFileAndContinue** API (which applies only to Windows Phone 8.1).

C#

```
#if WINDOWS_APP
    // Use Windows.Storage.Pickers.FileOpenPicker.PickSingleFileAsync
#else
    // Use Windows.Storage.Pickers.FileOpenPicker.PickSingleFileAndContinue
#endif // WINDOWS_APP
```

Windows 10 converges on the **PickSingleFileAsync** API, so your code simplifies to this:

C#

```
    // Use Windows.Storage.Pickers.FileOpenPicker.PickSingleFileAsync
```

In this example, we handle the hardware back button—but only on Windows Phone.

C#

```
#if WINDOWS_PHONE_APP
        Windows.Phone.UI.Input.HardwareButtons.BackPressed +=
this.HardwareButtons_BackPressed;
#endif // WINDOWS_PHONE_APP

...

#if WINDOWS_PHONE_APP
    void HardwareButtons_BackPressed(object sender,
Windows.Phone.UI.Input.BackPressedEventArgs e)
    {
        // Handle the event.
    }
#endif // WINDOWS_PHONE_APP
```

In Windows 10, the back button event is a universal concept. Back buttons implemented in hardware or in software will all raise the **BackRequested** event, so that's the one to handle.

C#

```
Windows.UI.Core.SystemNavigationManager.GetForCurrentView().BackRequested +=
        this.ViewModelLocator_BackRequested;

...

private void ViewModelLocator_BackRequested(object sender,
Windows.UI.Core.BackRequestedEventArgs e)
{
    // Handle the event.
}
```

This final example is similar to the previous one. Here, we handle the hardware camera button—but again, only in the code compiled into the Windows Phone app package.

C#

```
#if WINDOWS_PHONE_APP
    Windows.Phone.UI.Input.HardwareButtons.CameraPressed +=
this.HardwareButtons_CameraPressed;
```

```
#endif // WINDOWS_PHONE_APP

...

#if WINDOWS_PHONE_APP
void HardwareButtons_CameraPressed(object sender,
Windows.Phone.UI.Input.CameraEventArgs e)
{
    // Handle the event.
}
#endif // WINDOWS_PHONE_APP
```

In Windows 10, the hardware camera button is a concept particular to the mobile device family. Because one app package will be running on all devices, we change our compile-time condition into a run-time condition using what is known as adaptive code. To do that, we use the **ApiInformation** class to query at run-time for the presence of the **HardwareButtons** class. **HardwareButtons** is defined in the mobile extension SDK, so we'll need to add a reference to that SDK to our project for this code to compile. Note, though, that the handler will only be executed on a device that implements the types defined in the mobile extension SDK, and that's the mobile device family. So, this code is morally equivalent to the Universal 8.1 code in that it is careful only to use features that are present, although it achieves that in a different way.

C#

```
    // Note: Cache the value instead of querying it more than once.
    bool isHardwareButtonsAPIPresent =
Windows.Foundation.Metadata.ApiInformation.IsTypePresent
        ("Windows.Phone.UI.Input.HardwareButtons");

    if (isHardwareButtonsAPIPresent)
    {
        Windows.Phone.UI.Input.HardwareButtons.CameraPressed +=
            this.HardwareButtons_CameraPressed;
    }

    ...

private void HardwareButtons_CameraPressed(object sender,
Windows.Phone.UI.Input.CameraEventArgs e)
{
    // Handle the event.
}
```

Also, see Detecting the platform your app is running on.

App package manifest

The What's changed in Windows 10 topic lists changes to the package manifest schema reference for Windows 10, including elements that have been added, removed, and changed. For reference info on all elements, attributes, and types in the schema, see Element Hierarchy. If you're porting a Windows Phone Store app, or if your app is an update to an app from the Windows Phone Store, ensure that the **mp:PhoneIdentity** element matches what's in the app manifest of your previous app (use the same GUIDs that were assigned to the app by the Store). This will ensure that users of your app who are upgrading to Windows 10 or Windows 11 will receive your new app as an update, not a duplicate. See the **mp:PhoneIdentity** reference topic for more details.

The settings in your project (including any extension SDKs references) determine the API surface area that your app can call. But, your app package manifest is what determines the actual set of devices that your customers can install your app onto from the Store. For more info, see examples in **TargetDeviceFamily**.

You can edit the app package manifest to set various declarations, capabilities, and other settings that some features need. You can use the Visual Studio app package manifest editor to edit it. If the **Solution Explorer** is not shown, choose it from the **View** menu. Double-click **Package.appxmanifest**. This opens the manifest editor window. Select the appropriate tab to make changes and then save.

The next topic is Troubleshooting.

Related topics

- Develop apps for the Universal Windows Platform
- Jumpstart your Windows Runtime 8.x app using templates (C#, C++, Visual Basic)
- Creating Windows Runtime components
- Cross-Platform Development with the Portable Class Library

Troubleshooting porting Windows Runtime 8.x to UWP

Article • 10/20/2022

The previous topic was Porting the project.

We highly recommend reading to the end of this porting guide, but we also understand that you're eager to forge ahead and get to the stage where your project builds and runs. To that end, you can make temporary progress by commenting or stubbing out any non-essential code, and then returning to pay off that debt later. The table of troubleshooting symptoms and remedies in this topic may be helpful to you at this stage, although it's not a substitute for reading the next few topics. You can always refer back to the table as you progress through the later topics.

Tracking down issues

XAML parse exceptions can be difficult to diagnose, particularly if there are no meaningful error messages within the exception. Make sure that the debugger is configured to catch first-chance exceptions (to try and catch the parsing exception early on). You may be able to inspect the exception variable in the debugger to determine whether the HRESULT or message has any useful information. Also, check Visual Studio's output window for error messages output by the XAML parser.

If your app terminates and all you know is that an unhandled exception was thrown during XAML markup parsing, then that could be the result of a reference to a missing resource (that is, a resource whose key exists for Universal 8.1 apps but not for Windows 10 apps, such as some system **TextBlock** Style keys). Or it could be an exception thrown inside a **UserControl**, a custom control, or a custom layout panel.

A last resort is a binary split. Remove about half of the markup from a Page and re-run the app. You will then know whether the error is somewhere inside the half you removed (which you should now restore in any case) or in the half you did *not* remove. Repeat the process by splitting the half that contains the error, and so on, until you've zeroed in on the issue.

TargetPlatformVersion

This section explains what to do if, on opening a Windows 10 project in Visual Studio, you see the message "Visual Studio update required. One or more projects require a

platform SDK <version> that is either not installed or is included as part of a future update to Visual Studio."

- First, determine the version number of the SDK for Windows 10 that you have installed. Navigate to **C:\Program Files (x86)\Windows Kits\10\Include\<versionfoldername>** and make a note of <*versionfoldername*>, which will be in quad notation, "Major.Minor.Build.Revision".
- Open your project file for edit and find the `TargetPlatformVersion` and `TargetPlatformMinVersion` elements. Edit them to look like this, replacing <*versionfoldername*> with the quad notation version number you found on disk:

XML

```
<TargetPlatformVersion><versionfoldername></TargetPlatformVersion>
 <TargetPlatformMinVersion><versionfoldername></TargetPlatformMinVersion>
```

Troubleshooting symptoms and remedies

The remedy information in the table is intended to give you enough info to unblock yourself. You'll find further details about each of these issues as you read through later topics.

Symptom	Remedy
On opening a Windows 10 project in Visual Studio, you see the message "Visual Studio update required. One or more projects require a platform SDK <version> that is either not installed or is included as part of a future update to Visual Studio."	See the TargetPlatformVersion section in this topic.
A System.InvalidCastException is thrown when InitializeComponent is called in a xaml.cs file.	This can happen when you have more than one xaml file (at least one of which is MRT-qualified) sharing the same xaml.cs file and elements have x:Name attributes that are inconsistent between the two xaml files. Try adding the same name to the same elements in both xaml files, or omit names altogether.

Symptom	Remedy
When run on the device, the app terminates, or when launched from Visual Studio, you see the error “Unable to activate Windows Runtime 8.x app [...]. The activation request failed with error ‘Windows was unable to communicate with the target application. This usually indicates that the target application’s process aborted. [...]”.	The problem could be the imperative code running in your own Pages or in bound properties (or other types) during initialization. Or it could be happening while parsing the XAML file about to be displayed when the app terminated (if launching from Visual Studio, that will be the startup page). Look for invalid resource keys, and/or try some of the guidance in the "Tracking down issues" section in this topic.
The XAML parser or compiler, or a runtime exception, gives the error "*The resource "<resourcekey>" could not be resolved.*".	The resource key doesn't apply to Universal Windows Platform (UWP) apps (this is the case with some Windows Phone resources, for example). Find the correct equivalent resource and update your markup. Examples you might encounter right away are system keys such as `PhoneAccentBrush`.
The C# compiler gives the error "*The type or namespace name '<name>' could not be found [...]*" or "*The type or namespace name '<name>' does not exist in the namespace [...]*" or "*The type or namespace name '<name>' does not exist in the current context*".	This is likely to mean that type is implemented in an extension SDK (although there may be cases where the remedy is not so straightforward). Use the Windows APIs reference content to determine what extension SDK implements the API and then use Visual Studio's **Add** > **Reference** command to add a reference to that SDK to your project. If your app targets the set of APIs known as the universal device family then it's vital that you use the **ApiInformation** class to test at runtime for the presence of extension SDK before you call them (this is called adaptive code). If a universal API exists, then that's always preferable to an API in an extension SDK. For more info, see Extension SDKs.

The next topic is Porting XAML and UI.

Porting Windows Runtime 8.x XAML and UI to UWP

Article • 03/16/2023

The previous topic was Troubleshooting.

The practice of defining UI in the form of declarative XAML markup translates extremely well from Universal 8.1 apps to Universal Windows Platform (UWP) apps. You'll find that most of your markup is compatible, although you may need to make some adjustments to the system Resource keys or custom templates that you're using. The imperative code in your view models will require little or no change. Even much, or most, of the code in your presentation layer that manipulates UI elements should also be straightforward to port.

Imperative code

If you just want to get to the stage where your project builds, you can comment or stub out any non-essential code. Then iterate, one issue at a time, and refer to the following topics in this section (and the previous topic: Troubleshooting), until any build and runtime issues are ironed-out and your port is complete.

Adaptive/responsive UI

Because your app can run on a potentially wide range of devices—each with its own screen size and resolution—you'll want to go beyond the minimal steps to port your app and you'll want to tailor your UI to look its best on those devices. You can use the adaptive Visual State Manager feature to dynamically detect window size and to change layout in response, and an example of how to do that is shown in the section Adaptive UI in the Bookstore2 case study topic.

Back button handling

For Universal 8.1 apps, Windows Runtime 8.x apps and Windows Phone Store apps have different approaches to the UI you show and the events you handle for the back button. But, for Windows 10 apps, you can use a single approach in your app. On mobile devices, the button is provided for you as a capacitive button on the device, or as a button in the shell. On a desktop device, you add a button to your app's chrome whenever back-navigation is possible within the app, and this appears in the title bar for

windowed apps or in the task bar for Tablet mode (Windows 10 only). The back button event is a universal concept across all device families, and buttons implemented in hardware or in software raise the same **BackRequested** event.

The example below works for all device families and it is good for cases where the same processing applies to all pages, and where you do not need to confirm navigation (for example, to warn about unsaved changes).

C#

```
   // app.xaml.cs

    protected override void OnLaunched(LaunchActivatedEventArgs e)
    {
        [...]


Windows.UI.Core.SystemNavigationManager.GetForCurrentView().BackRequested +=
App_BackRequested;
        rootFrame.Navigated += RootFrame_Navigated;
    }

    private void RootFrame_Navigated(object sender, NavigationEventArgs e)
    {
        Frame rootFrame = Window.Current.Content as Frame;

        // Note: On device families that have no title bar, setting
AppViewBackButtonVisibility can safely execute
        // but it will have no effect. Such device families provide back
button UI for you.
        if (rootFrame.CanGoBack)
        {

Windows.UI.Core.SystemNavigationManager.GetForCurrentView().AppViewBackButto
nVisibility =
                Windows.UI.Core.AppViewBackButtonVisibility.Visible;
        }
        else
        {

Windows.UI.Core.SystemNavigationManager.GetForCurrentView().AppViewBackButto
nVisibility =
                Windows.UI.Core.AppViewBackButtonVisibility.Collapsed;
        }
    }

    private void App_BackRequested(object sender,
Windows.UI.Core.BackRequestedEventArgs e)
    {
        Frame rootFrame = Window.Current.Content as Frame;

        if (rootFrame.CanGoBack)
```

```
        {
            rootFrame.GoBack();
        }
    }
```

There's also a single approach for all device families for programmatically exiting the app.

C#

```
Windows.UI.Xaml.Application.Current.Exit();
```

Charms

You don't need to change any of your code that integrates with charms, but you do need to add some UI to your app to take the place of the Charms bar, which is not a part of the Windows 10 shell. A Universal 8.1 app running on Windows 10 has its own replacement UI provided by system-rendered chrome in the app's title bar.

Controls, and control styles and templates

A Universal 8.1 app running on Windows 10 will retain the 8.1 appearance and behavior with respect to controls. But, when you port that app to a Windows 10 app, there are some differences in appearance and behavior to be aware of. The architecture and design of controls is essentially unchanged for Windows 10 apps, so the changes are mostly around the design language, simplification, and usability improvements.

Note The PointerOver visual state is relevant in custom styles/templates in Windows 10 apps and in Windows Runtime 8.x apps, but not in Windows Phone Store apps. For this reason (and because of the system resource keys that are supported for Windows 10 apps), we recommend that you re-use the custom styles/templates from your Windows Runtime 8.x apps when you're porting your app to Windows 10. If you want to be certain that your custom styles/templates are using the latest set of visual states, and are benefitting from performance improvements made to the default styles/templates, then edit a copy of the new Windows 10 default template and re-apply your customization to that. One example of a performance improvement is that any **Border** that formerly enclosed a **ContentPresenter** or a Panel has been removed and a child element now renders the border.

Here are some more specific examples of changes to controls.

Control name	Change
AppBar	If you are using the **AppBar** control (CommandBar is recommended instead), then it is not hidden by default in a Windows 10 app. You can control this with the AppBar.ClosedDisplayMode property.
AppBar, CommandBar	In a Windows 10 app, **AppBar** and CommandBar have a **See more** button (the ellipsis).
CommandBar	In a Windows Runtime 8.x app, the secondary commands of a CommandBar are always visible. In a Windows Phone Store app, and in a Windows 10 app, the don't appear until the command bar opens.
CommandBar	For a Windows Phone Store app, the value of CommandBar.IsSticky does not affect whether or not the bar is light-dismissible. For a Windows 10 app, if **IsSticky** is set to true, then the **CommandBar** disregards a light dismiss gesture.
CommandBar	In a Windows 10 app, CommandBar does not handle the EdgeGesture.Completed nor UIElement.RightTapped events. Nor does it respond to a tap nor a swipe up. You still have the option to handle these events and set IsOpen.
DatePicker, TimePicker	Review how your app looks with the visual changes to DatePicker and TimePicker. For a Windows 10 app running on a mobile device, these controls no longer navigate to a selection page but instead use a light-dismissible popup.
DatePicker, TimePicker	In a Windows 10 app, you can't put DatePicker or TimePicker inside a fly-out. If you want those controls to be displayed in a popup-type control, then you can use DatePickerFlyout and TimePickerFlyout.
GridView, ListView	For **GridView/ListView**, see GridView and ListView changes.
Hub	In a Windows Phone Store app, a Hub control wraps around from the last section to the first. In a Windows Runtime 8.x app, and in a Windows 10 app, hub sections do not wrap around.
Hub	In a Windows Phone Store app, a Hub control's background image moves in parallax relative to the hub sections. In a Windows Runtime 8.x app, and in a Windows 10 app, parallax is not used.
Hub	In a Universal 8.1 app, the HubSection.IsHeaderInteractive property causes the section header—and a chevron glyph rendered next to it—to become interactive. In a Windows 10 app, there is an interactive "See more" affordance beside the header, but the header itself is not interactive. **IsHeaderInteractive** still determines whether interaction raises the Hub.SectionHeaderClick event.
MessageDialog	If you're using **MessageDialog**, then consider instead using the more flexible ContentDialog. Also, see the XAML UI Basics sample.

Control name	Change
ListPickerFlyout, **PickerFlyout**	**ListPickerFlyout** and **PickerFlyout** are deprecated for a Windows 10 app. For a single selection fly-out, use **MenuFlyout**; for more complex experiences, use **Flyout**.
PasswordBox	The **PasswordBox.IsPasswordRevealButtonEnabled** property is deprecated in a Windows 10 app, and setting it has no effect. Use **PasswordBox.PasswordRevealMode** instead, which defaults to **Peek** (in which an eye glyph is displayed, like in a Windows Runtime 8.x app). Also, see Guidelines for password boxes.
Pivot	The **Pivot** control is now universal, it is no longer limited to use on mobile devices.
SearchBox	Although **SearchBox** is implemented in the Universal device family, it is not fully functional on mobile devices. See SearchBox deprecated in favor of AutoSuggestBox.
SemanticZoom	For **SemanticZoom**, see SemanticZoom changes.
ScrollViewer	Some default properties of **ScrollViewer** have changed. **HorizontalScrollMode** is **Auto**, **VerticalScrollMode** is **Auto**, and **ZoomMode** is **Disabled**. If the new default values are not appropriate for your app, then you can change them either in a style or as local values on the control itself.
TextBox	In a Windows Runtime 8.x app, spell-checking is off by default for a **TextBox**. In a Windows Phone Store app, and in a Windows 10 app, it is on by default.
TextBox	The default font size for a **TextBox** has changed from 11 to 15.
TextBox	The default value of **TextBox.TextReadingOrder** has changed from **Default** to **DetectFromContent**. If that's undesirable, then use **UseFlowDirection**. **Default** is deprecated.
Various	Accent color applies to a Windows Phone Store apps, and to Windows 10 apps, but not to Windows Runtime 8.x apps.

For more info on UWP app controls, see Controls by function, Controls list, and Guidelines for controls.

Design language in Windows 10

There are some small but important differences in design language between Universal 8.1 apps and Windows 10 apps. For all the details, see Design. Despite the design language changes, our design principles remain consistent: be attentive to detail but always strive for simplicity through focusing on content not chrome, fiercely reducing visual elements, and remaining authentic to the digital domain; use visual hierarchy

especially with typography; design on a grid; and bring your experiences to life with fluid animations.

Effective pixels, viewing distance, and scale factors

Previously, view pixels were the way to abstract the size and layout of UI elements away from the actual physical size and resolution of devices. View pixels have now evolved into effective pixels, and here's an explanation of that term, what it means, and the extra value it offers.

The term "resolution" refers to a measure of pixel density and not, as is commonly thought, pixel count. "Effective resolution" is the way the physical pixels that compose an image or glyph resolve to the eye given differences in viewing distance and the physical pixel size of the device (pixel density being the reciprocal of physical pixel size). Effective resolution is a good metric to build an experience around because it is user-centric. By understanding all the factors, and controlling the size of UI elements, you can make the user's experience a good one.

Different devices are a different number of effective pixels wide, ranging from 320 epx for the smallest devices, to 1024 epx for a modest-sized monitor, and far beyond to much higher widths. All you have to do is continue to use auto-sized elements and dynamic layout panels as you always have. There will also be some cases where you'll set the properties of your UI elements to a fixed size in XAML markup. A scale factor is automatically applied to your app depending on what device it runs on and the display settings made by the user. And that scale factor serves to keep any UI element with a fixed size presenting a more-or-less constant-sized touch (and reading) target to the user across a wide variety of screen sizes. And together with dynamic layout, your UI won't merely optically scale on different devices. It will instead do what's necessary to fit the appropriate amount of content into the available space.

So that your app has the best experience across all displays, we recommend that you create each bitmap asset in a range of sizes, each suitable for a particular scale factor. Providing assets at 100%-scale, 200%-scale, and 400%-scale (in that priority order) will give you excellent results in most cases at all the intermediate scale factors.

Note If, for whatever reason, you cannot create assets in more than one size, then create 100%-scale assets. In Microsoft Visual Studio, the default project template for UWP apps provides branding assets (tile images and logos) in only one size, but they are not 100%-scale. When authoring assets for your own app, follow the guidance in this section and provide 100%, 200%, and 400% sizes, and use asset packs.

If you have intricate artwork, then you may want to provide your assets in even more sizes. If you're starting with vector art, then it's relatively easy to generate high-quality assets at any scale factor.

We don't recommend that you try to support all of the scale factors, but the full list of scale factors for Windows 10 apps is 100%, 125%, 150%, 200%, 250%, 300%, and 400%. If you provide them, the Store will pick the correct-sized asset(s) for each device, and only those assets will be downloaded. The Store selects the assets to download based on the DPI of the device. You can re-use assets from your Windows Runtime 8.x app at scale factors such as 140% and 220%, but your app will run at one of the new scale factors and so some bitmap scaling will be unavoidable. Test your app on a range of devices to see whether you're happy with the results in your case.

You may be re-using XAML markup from a Windows Runtime 8.x app where literal dimension values are used in the markup (perhaps to size shapes or other elements, perhaps for typography). But, in some cases, a larger scale factor is used on a device for a Windows 10 app than for a Universal 8.1 app (for example, 150% is used where 140% was before, and 200% is used where 180% was). So, if you find that these literal values are now too big on Windows 10, then try multiplying them by 0.8. For more info, see Responsive design 101 for UWP apps.

GridView and ListView changes

Several changes have been made to the default style setters for **GridView** to make the control scroll vertically (instead of horizontally, as it did previously by default). If you edited a copy of the default style in your project, then your copy won't have these changes, so you'll need to make them manually. Here is a list of the changes.

- The setter for **ScrollViewer.HorizontalScrollBarVisibility** has changed from **Auto** to **Disabled**.
- The setter for **ScrollViewer.VerticalScrollBarVisibility** has changed from **Disabled** to **Auto**.
- The setter for **ScrollViewer.HorizontalScrollMode** has changed from **Enabled** to **Disabled**.
- The setter for **ScrollViewer.VerticalScrollMode** has changed from **Disabled** to **Enabled**.
- In the setter for **ItemsPanel**, the value of **ItemsWrapGrid.Orientation** has changed from **Vertical** to **Horizontal**.

If that last change (the change to **Orientation**) seems contradictory, then remember that we're talking about a wrap grid. A horizontally-oriented wrap grid (the new value) is similar to a writing system where text flows horizontally and breaks to the next line

down at the end of a page. A page of text like that scrolls vertically. Conversely, a vertically-oriented wrap grid (the previous value) is similar to a writing system where text flows vertically and therefore scrolls horizontally.

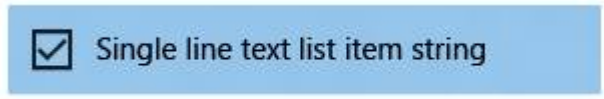
Here are the aspects of GridView and ListView that have change or are not supported in Windows 10.

- The IsSwipeEnabled property (Windows Runtime 8.x apps only) is not supported for Windows 10 apps. The API is still present, but setting it has no effect. All previous selection gestures are supported except downward swipe (which is unsupported because data shows that it is not discoverable) and right-click (which is reserved for showing a context menu).
- The ReorderMode property (Windows Phone Store apps only) is not supported for Windows 10 apps. The API is still present, but setting it has no effect. Instead, set AllowDrop and CanReorderItems to true on your **GridView** or **ListView** and then the user will be able to reorder using a press-and-hold (or click-and-drag) gesture.
- When developing for Windows 10, use ListViewItemPresenter instead of GridViewItemPresenter in your item container style, both for ListView and for GridView. If you edit a copy of the default item container styles, then you will get the correct type.
- The selection visuals have changed for a Windows 10 app. If you set SelectionMode to **Multiple**, then by default, a check box is rendered for each item. The default setting for **ListView** items means that the check box is laid out inline beside the item, and as a result, the space occupied by the rest of the item will be slightly reduced and shifted. For **GridView** items, the check box is overlaid on top of the item by default. But, in either case, you can control the layout (Inline or Overlay) of the check boxes (with the CheckMode property) and whether they are shown at all (with the SelectionCheckMarkVisualEnabled property) on the ListViewItemPresenter element inside your item container style as in the example below.
- In Windows 10, the ContainerContentChanging event is raised twice per item during UI virtualization: once for the reclaim, and once for the re-use. If the value of InRecycleQueue is **true** and you have no special reclaim work to do, then you can exit your event handler immediately with the assurance that it will be re-entered when that same item is re-used (at which time **InRecycleQueue** will be **false**).

XML

```
<Style x:Key="CustomItemContainerStyle"
TargetType="ListViewItem|GridViewItem">
    ...
    <Setter.Value>
```

```
        <ControlTemplate TargetType="ListViewItem|GridViewItem">
            <ListViewItemPresenter CheckMode="Inline|Overlay" ... />
        </ControlTemplate>
    </Setter.Value>
    ...
</Style>
```



A ListViewItemPresenter with inline check box

A ListViewItemPresenter with an overlaid check box

- With the removal of downward swipe and right-click gestures for selection (for the reasons given above), the interaction model has changed, one consequence of which is that the **ItemClick** and **SelectionChanged** events are no longer mutually exclusive. For your Windows 10 app, review your scenarios and decide whether to adopt the "selection" or the "invoke" interaction model. For details, see How to change the interaction mode.
- There are some changes to the properties that you use to style **ListViewItemPresenter**. Properties that are new are **CheckBoxBrush**, **PressedBackground**, **SelectedPressedBackground**, and **FocusSecondaryBorderBrush**. Properties that are ignored for a Windows 10 app are **Padding** (use **ContentMargin** instead), **CheckHintBrush**, **CheckSelectingBrush**, **PointerOverBackgroundMargin**, **ReorderHintOffset**, **SelectedBorderThickness**, and **SelectedPointerOverBorderBrush**.

This table describes the changes to the visual states and visual state groups in the **ListViewItem** and **GridViewItem** control templates.

8.1	Feature state	Windows 10/11	Feature state
CommonStates		CommonStates	
	Normal		Normal
	PointerOver		PointerOver
	Pressed		Pressed

8.1	Feature state	Windows 10/11	Feature state
	PointerOverPressed		[unavailable]
	Disabled		[unavailable]
	[unavailable]		PointerOverSelected
	[unavailable]		Selected
	[unavailable]		PressedSelected
[unavailable]		DisabledStates	
	[unavailable]		Disabled
	[unavailable]		Enabled
SelectionHintStates		[unavailable]	
	VerticalSelectionHint		[unavailable]
	HorizontalSelectionHint		[unavailable]
	NoSelectionHint		[unavailable]
[unavailable]		MultiSelectStates	
	[unavailable]		MultiSelectDisabled
	[unavailable]		MultiSelectEnabled
SelectionStates		[unavailable]	
	Unselecting		[unavailable]
	Unselected		[unavailable]
	UnselectedPointerOver		[unavailable]
	UnselectedSwiping		[unavailable]
	Selecting		[unavailable]
	Selected		[unavailable]
	SelectedSwiping		[unavailable]
	SelectedUnfocused		[unavailable]

If you have a custom **ListViewItem** or **GridViewItem** control template, then review it in light of the above changes. We recommend that you start over by editing a copy of the new default template and re-applying your customization to that. If, for whatever

reason, you can't do that and you need to edit your existing template, then here is some general guidance around how you might go about doing that.

- Add the new MultiSelectStates visual state group.
- Add the new MultiSelectDisabled visual state.
- Add the new MultiSelectEnabled visual state.
- Add the new DisabledStates visual state group.
- Add the new Enabled visual state.
- In the CommonStates visual state group, remove the PointerOverPressed visual state.
- Move the Disabled visual state to the DisabledStates visual state group.
- Add the new PointerOverSelected visual state.
- Add the new PressedSelected visual state.
- Remove the SelectedHintStates visual state group.
- In the SelectionStates visual state group, move the Selected visual state to the CommonStates visual state group.
- Remove the entire SelectionStates visual state group.

Localization and globalization

You can re-use the Resources.resw files from your Universal 8.1 project in your UWP app project. After copying the file over, add it to the project and set **Build Action** to **PRIResource** and **Copy to Output Directory** to **Do not copy**. The ResourceContext.QualifierValues topic describes how to load device family-specific resources based on the device family resource selection factor.

Play To

The APIs in the Windows.Media.PlayTo namespace are deprecated for Windows 10 apps in favor of the Windows.Media.Casting APIs.

Resource keys, and TextBlock style sizes

The design language has evolved for Windows 10 and consequently certain system styles have changed. In some cases, you will want to revisit the visual designs of your views so that they are in harmony with the style properties that have changed.

In other cases, resource keys are no longer supported. The XAML markup editor in Visual Studio highlights references to resource keys that can't be resolved. For example, the XAML markup editor will underline a reference to the style key

`ListViewItemTextBlockStyle` with a red squiggle. If that isn't corrected, then the app will immediately terminate when you try to deploy it to the emulator or device. So, it's important to attend to XAML markup correctness. And you will find Visual Studio to be a great tool for catching such issues.

For keys that are still supported, changes in design language mean that properties set by some styles have changed. For example, `TitleTextBlockStyle` sets **FontSize** to 14.667px in a Windows Runtime 8.x app and 18.14px in a Windows Phone Store app. But, the same style sets **FontSize** to a much larger 24px in a Windows 10 app. Review your designs and layouts and use the appropriate styles in the right places. For more info, see Guidelines for fonts and Design UWP apps.

This is a full list of the keys that are no longer supported.

- CheckBoxAndRadioButtonMinWidthSize
- CheckBoxAndRadioButtonTextPaddingThickness
- ComboBoxFlyoutListPlaceholderTextOpacity
- ComboBoxFlyoutListPlaceholderTextThemeMargin
- ComboBoxHighlightedBackgroundThemeBrush
- ComboBoxHighlightedBorderThemeBrush
- ComboBoxHighlightedForegroundThemeBrush
- ComboBoxInlinePlaceholderTextForegroundThemeBrush
- ComboBoxInlinePlaceholderTextThemeFontWeight
- ComboBoxItemDisabledThemeOpacity
- ComboBoxItemHighContrastBackgroundThemeMargin
- ComboBoxItemMinHeightThemeSize
- ComboBoxPlaceholderTextBlockStyle
- ComboBoxPlaceholderTextThemeMargin
- CommandBarBackgroundThemeBrush
- CommandBarForegroundThemeBrush
- ContentDialogButton1HostPadding
- ContentDialogButton2HostPadding
- ContentDialogButtonsMinHeight
- ContentDialogContentLandscapeWidth
- ContentDialogContentMinHeight
- ContentDialogDimmingColor
- ContentDialogTitleMinHeight
- ControlContextualInfoTextBlockStyle
- ControlHeaderContentPresenterStyle
- ControlHeaderTextBlockStyle
- FlyoutContentPanelLandscapeThemeMargin

- FlyoutContentPanelPortraitThemeMargin
- GrabberMargin
- GridViewItemMargin
- GridViewItemPlaceholderBackgroundThemeBrush
- GroupHeaderTextBlockStyle
- HeaderContentPresenterStyle
- HighContrastBlack
- HighContrastWhite
- HubHeaderCharacterSpacing
- HubHeaderFontSize
- HubHeaderMarginThickness
- HubSectionHeaderCharacterSpacing
- HubSectionHeaderFontSize
- HubSectionHeaderMarginThickness
- HubSectionMarginThickness
- InlineWindowPlayPauseMargin
- ItemTemplate
- LeftFullWindowMargin
- LeftMargin
- ListViewEmptyStaticTextBlockStyle
- ListViewItemContentTextBlockStyle
- ListViewItemContentTranslateX
- ListViewItemMargin
- ListViewItemMultiselectCheckBoxMargin
- ListViewItemSubheaderTextBlockStyle
- ListViewItemTextBlockStyle
- MediaControlPanelAudioThemeBrush
- MediaControlPanelPhoneVideoThemeBrush
- MediaControlPanelVideoThemeBrush
- MediaControlPanelVideoThemeColor
- MediaControlPlayPauseThemeBrush
- MediaControlTimeRowThemeBrush
- MediaControlTimeRowThemeColor
- MediaDownloadProgressIndicatorThemeBrush
- MediaErrorBackgroundThemeBrush
- MediaTextThemeBrush
- MenuFlyoutBackgroundThemeBrush
- MenuFlyoutBorderThemeBrush
- MenuFlyoutLandscapeThemePadding
- MenuFlyoutLeftLandscapeBorderThemeThickness

- MenuFlyoutPortraitBorderThemeThickness
- MenuFlyoutPortraitThemePadding
- MenuFlyoutRightLandscapeBorderThemeThickness
- MessageDialogContentStyle
- MessageDialogTitleStyle
- MinimalWindowMargin
- PasswordBoxCheckBoxThemeMargin
- PhoneAccentBrush
- PhoneBackgroundBrush
- PhoneBackgroundColor
- PhoneBaseBlackColor
- PhoneBaseHighColor
- PhoneBaseLowColor
- PhoneBaseLowSolidColor
- PhoneBaseMediumHighColor
- PhoneBaseMediumMidColor
- PhoneBaseMediumMidSolidColor
- PhoneBaseMidColor
- PhoneBaseWhiteColor
- PhoneBorderThickness
- PhoneButtonBasePressedForegroundBrush
- PhoneButtonContentPadding
- PhoneButtonFontWeight
- PhoneButtonMinHeight
- PhoneButtonMinWidth
- PhoneChromeBrush
- PhoneChromeColor
- PhoneControlBackgroundColor
- PhoneControlDisabledColor
- PhoneControlForegroundColor
- PhoneDisabledBrush
- PhoneDisabledColor
- PhoneFontFamilyLight
- PhoneFontFamilySemiBold
- PhoneForegroundBrush
- PhoneForegroundColor
- PhoneHighContrastSelectedBackgroundThemeBrush
- PhoneHighContrastSelectedForegroundThemeBrush
- PhoneImagePlaceholderColor
- PhoneLowBrush

- PhoneMidBrush
- PhonePageBackgroundColor
- PhonePivotLockedTranslation
- PhonePivotUnselectedItemOpacity
- PhoneRadioCheckBoxBorderBrush
- PhoneRadioCheckBoxBrush
- PhoneRadioCheckBoxCheckBrush
- PhoneRadioCheckBoxPressedBrush
- PhoneStrokeThickness
- PhoneTextHighColor
- PhoneTextLowColor
- PhoneTextMidColor
- PhoneTextOverAccentColor
- PhoneTouchTargetLargeOverhang
- PhoneTouchTargetOverhang
- PivotHeaderItemPadding
- PlaceholderContentPresenterStyle
- ProgressBarHighContrastAccentBarThemeBrush
- ProgressBarIndeterminateRectagleThemeSize
- ProgressBarRectangleStyle
- ProgressRingActiveBackgroundOpacity
- ProgressRingElipseThemeMargin
- ProgressRingElipseThemeSize
- ProgressRingTextForegroundThemeBrush
- ProgressRingTextThemeMargin
- ProgressRingThemeSize
- RichEditBoxTextThemeMargin
- RightFullWindowMargin
- RightMargin
- ScrollBarMinThemeHeight
- ScrollBarMinThemeWidth
- ScrollBarPanningThumbThemeHeight
- ScrollBarPanningThumbThemeWidth
- SliderThumbDisabledBorderThemeBrush
- SliderTrackBorderThemeBrush
- SliderTrackDisabledBorderThemeBrush
- TextBoxBackgroundColor
- TextBoxBorderColor
- TextBoxDisabledHeaderForegroundThemeBrush
- TextBoxFocusedBackgroundThemeBrush

- TextBoxForegroundColor
- TextBoxPlaceholderColor
- TextControlHeaderMarginThemeThickness
- TextControlHeaderMinHeightSize
- TextStyleExtraExtraLargeFontSize
- TextStyleExtraLargePlusFontSize
- TextStyleMediumFontSize
- TextStyleSmallFontSize
- TimeRemainingElementMargin

SearchBox deprecated in favor of AutoSuggestBox

Although SearchBox is implemented in the Universal device family, it is not fully functional on mobile devices. Use AutoSuggestBox for your universal search experience. Here's how you typically implement a search experience with **AutoSuggestBox**.

Once the user starts typing, the **TextChanged** event is raised, with a reason of **UserInput**. You then populate the list of suggestions and set the **ItemsSource** of the AutoSuggestBox. As the user navigates the list, the **SuggestionChosen** event is raised (and if you have set **TextMemberDisplayPath**, the text box is auto-filled with the property specified). When the user submits a choice with the Enter key, the **QuerySubmitted** event is raised, at which point you can take action on that suggestion (in this case, most likely navigating to another page with more details on the specified content). Note that the **LinguisticDetails** and **Language** properties of **SearchBoxQuerySubmittedEventArgs** are no longer supported (there are equivalent APIs to support that functionality). And **KeyModifiers** is no longer supported.

AutoSuggestBox also has support for input method editors (IMEs). And, if you want to show a "find" icon, then you can do that too (interacting with the icon will cause the **QuerySubmitted** event to be raised).

XML

```
<AutoSuggestBox ... >
     <AutoSuggestBox.QueryIcon>
         <SymbolIcon Symbol="Find"/>
     </AutoSuggestBox.QueryIcon>
 </AutoSuggestBox>
```

Also, see AutoSuggestBox porting sample .

SemanticZoom changes

The zooming-out gesture for a **SemanticZoom** has converged on the Windows Phone model, which is to tap or click a group header (so, on desktop computers, the minus button affordance to zoom out is no longer displayed). Now, we get the same, consistent, behavior for free on all devices. One cosmetic difference from the Windows Phone model is that the zoomed-out view (the jump list) replaces the zoomed-in view rather than overlaying it. For this reason, you can remove any semi-opaque backgrounds from zoomed-out views.

In a Windows Phone Store app, the zoomed-out view expands to the size of the screen. In a Windows Runtime 8.x app, and in a Windows 10 app, the size of the zoomed-out view is constrained to the bounds of the **SemanticZoom** control.

In a Windows Phone Store app, content behind the zoomed-out view (in z-order) shows through if the zoomed-out view has any transparency in its background. In a Windows Runtime 8.x app, and in a Windows 10 app, nothing is visible behind the zoomed out view.

In a Windows Runtime 8.x app, when the app is deactivated and reactivated, the zoomed-out view is dismissed (if it was being shown) and the zoomed-in view is shown instead. In a Windows Phone Store app, and in a Windows 10 app, the zoomed-out view will remain showing if it was being shown.

In a Windows Phone Store app, and in a Windows 10 app, the zoomed-out view is dismissed when the back button is pressed. For a Windows Runtime 8.x app, there is no built-in back button processing, so the question doesn't apply.

Settings

The Windows Runtime 8.x **SettingsPane** class is not appropriate for Windows 10. Instead, in addition to building a Settings page, you should give your users a way to access it from within your app. We recommend that you expose this app Settings page at the top level, as the last pinned item on your navigation pane, but here are the full set of your options.

- Navigation pane. Settings should be the last item in the navigational list of choices, and pinned to the bottom.
- Appbar/toolbar (within a tabs view or pivot layout). Settings should be the last item in the appbar or toolbar menu flyout. It is not recommended for Settings to be one of the top-level items within the navigation.

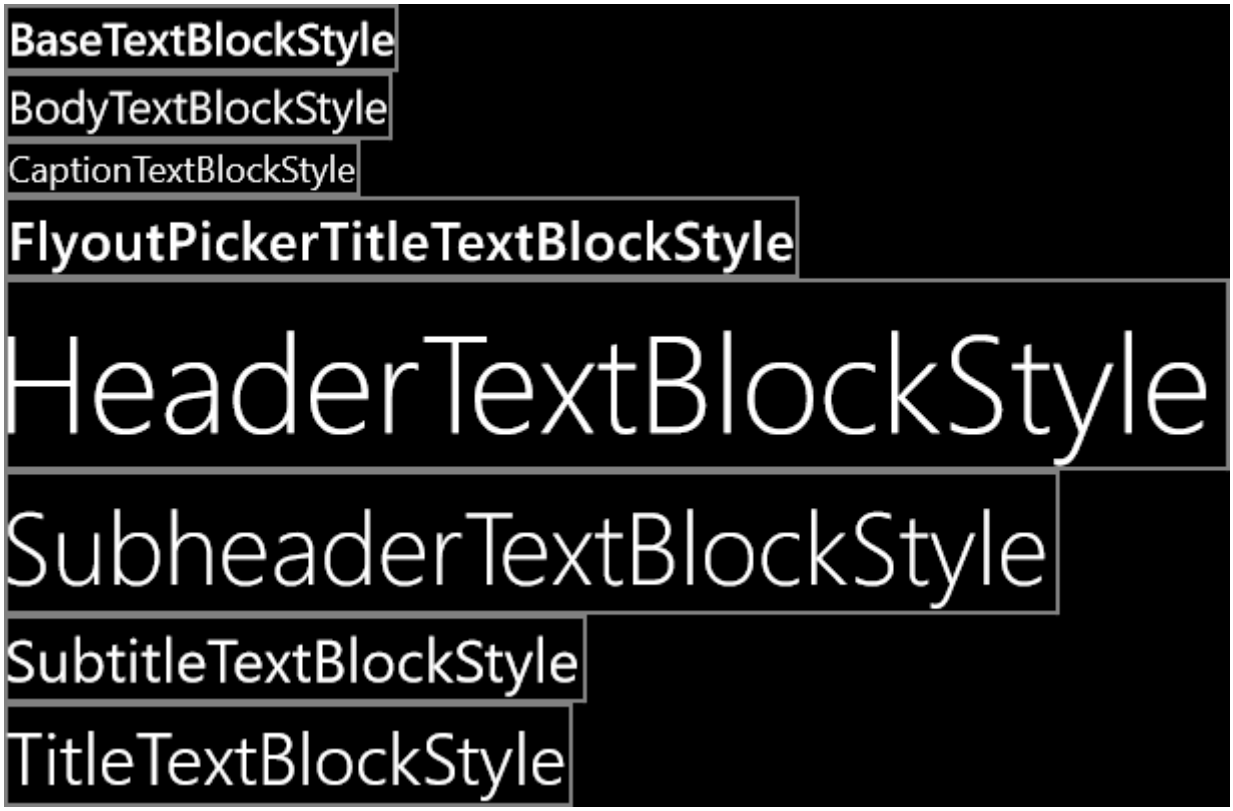
- Hub. Settings should be located inside of the menu flyout (could be from the app bar menu or the toolbar menu within the Hub layout).

It's also not recommended to bury Settings within a master-detail pane.

Your Settings page should fill the whole of your app's window, and your Settings page is also where About and Feedback should be. For guidance on the design of your Settings page, see Guidelines for app settings.

Text

Text (or typography) is an important aspect of a UWP app and, while porting, you may want to revisit the visual designs of your views so that they are in harmony with the new design language. Use these illustrations to find the Universal Windows Platform (UWP) **TextBlock** system styles that are available. Find the ones that correspond to the Windows Phone Silverlight styles you used. Alternatively, you can create your own universal styles and copy the properties from the Windows Phone Silverlight system styles into those.



System TextBlock styles for Windows 10 apps

In Windows Runtime 8.x apps and Windows Phone Store apps, the default font family is Global User Interface. In a Windows 10 app, the default font family is Segoe UI. As a result, font metrics in your app may look different. If you want to reproduce the look of your 8.1 text, you can set your own metrics using properties such as **LineHeight** and **LineStackingStrategy**.

In Windows Runtime 8.x apps and Windows Phone Store apps, the default language for text is set to the language of the build, or to en-us. In a Windows 10 app, the default language is set to the top app language (font fallback). You can set **FrameworkElement.Language** explicitly, but you will enjoy better font fallback behavior if you do not set a value for that property.

For more info, see Guidelines for fonts and Design UWP apps. Also, see the Controls section above for changes to text controls.

Theme changes

For a Universal 8.1 app, the default theme is dark by default. For Windows 10 devices, the default theme has changed, but you can control the theme used by declaring a requested theme in App.xaml. For example, to use a dark theme on all devices, add `RequestedTheme="Dark"` to the root Application element.

Tiles and toasts

For tiles and toasts, the templates you're currently using will continue to work in your Windows 10 app. But, there are new, adaptive templates available for you to use, and these are described in Notifications, tiles, toasts, and badges.

Previously, on desktop computers, a toast notification was a transitory message. It would disappear, and no longer be retrievable, once it was missed or ignored. On Windows Phone, if a toast notification is ignored or temporarily dismissed, it would go into the Action Center. Now, Action Center is no longer limited to the Mobile device family.

To send a toast notification, there is no longer any need to declare a capability.

Window size

For a Universal 8.1 app, the **ApplicationView** app manifest element is used to declare a minimum window width. In your UWP app, you can specify a minimum size (both width and height) with imperative code. The default minimum size is 500x320epx, and that's also the smallest minimum size accepted. The largest minimum size accepted is 500x500epx.

```
C#

Windows.UI.ViewManagement.ApplicationView.GetForCurrentView().SetPreferredMi
```

```
nSize
        (new Size { Width = 500, Height = 500 });
```

The next topic is Porting for I/O, device, and app model.

Porting Windows Runtime 8.x to UWP for I/O, device, and app model

Article • 10/20/2022

The previous topic was Porting XAML and UI.

Code that integrates with the device itself and its sensors involves input from, and output to, the user. It can also involve processing data. But, this code is not generally thought of as either the UI layer *or* the data layer. This code includes integration with the vibration controller, accelerometer, gyroscope, microphone and speaker (which intersect with speech recognition and synthesis), (geo)location, and input modalities such as touch, mouse, keyboard, and pen.

Application lifecycle (process lifetime management)

For a Universal 8.1 app, there is a two-second "debounce window" of time between the app becoming inactive and the system raising the suspending event. Using this debounce window as extra time to suspend state is unsafe, and for a Universal Windows Platform (UWP) app, there is no debounce window at all; the suspension event is raised as soon as an app becomes inactive.

For more info, see App lifecycle.

Background audio

For the **MediaElement.AudioCategory** property, **ForegroundOnlyMedia** and **BackgroundCapableMedia** are deprecated for Windows 10 apps. Use the Windows Phone Store app model instead. For more information, see Background Audio.

Detecting the platform your app is running on

The way of thinking about app-targeting changes with Windows 10. The new conceptual model is that an app targets the Universal Windows Platform (UWP) and runs across all Windows devices. It can then opt to light up features that are exclusive to particular device families. If needed, the app also has the option to limit itself to targeting one or more device families specifically. For more info on what device families are—and how to decide which device family to target—see Guide to UWP apps.

If you have code in your Universal 8.1 app that detects what operating system it is running on, then you may need to change that depending on the reason for the logic. If the app is passing the value through, and not acting on it, then you may want to continue to collect the operating system info.

Note We recommend that you not use operating system or device family to detect the presence of features. Identifying the current operating system or device family is usually not the best way to determine whether a particular operating system or device family feature is present. Rather than detecting the operating system or device family (and version number), test for the presence of the feature itself (see Conditional compilation, and adaptive code). If you must require a particular operating system or device family, be sure to use it as a minimum supported version, rather than design the test for that one version.

To tailor your app's UI to different devices, there are several techniques that we recommend. Continue to use auto-sized elements and dynamic layout panels as you always have. In your XAML markup, continue to use sizes in effective pixels (formerly view pixels) so that your UI adapts to different resolutions and scale factors (see Effective pixels, viewing distance, and scale factors.). And use Visual State Manager's adaptive triggers and setters to adapt your UI to the window size (see Guide to UWP apps.).

However, if you have a scenario where it is unavoidable to detect the device family, then you can do that. In this example, we use the **AnalyticsVersionInfo** class to navigate to a page tailored for the mobile device family where appropriate, and we make sure to fall back to a default page otherwise.

C#

```
   if (Windows.System.Profile.AnalyticsInfo.VersionInfo.DeviceFamily ==
"Windows.Mobile")
        rootFrame.Navigate(typeof(MainPageMobile), e.Arguments);
    else
        rootFrame.Navigate(typeof(MainPage), e.Arguments);
```

Your app can also determine the device family that it is running on from the resource selection factors that are in effect. The example below shows how to do this imperatively, and the **ResourceContext.QualifierValues** topic describes the more typical use case for the class in loading device family-specific resources based on the device family factor.

C#

```
var qualifiers =
Windows.ApplicationModel.Resources.Core.ResourceContext.GetForCurrentView().
QualifierValues;
string deviceFamilyName;
bool isDeviceFamilyNameKnown = qualifiers.TryGetValue("DeviceFamily", out
deviceFamilyName);
```

Also, see Conditional compilation, and adaptive code.

Location

When an app that declares the Location capability in its app package manifest runs on Windows 10, the system will prompt the end-user for consent. This is true whether the app is a Windows Phone Store app or a Windows 10 app. So, if your app displays its own custom consent prompt, or if it provides an on-off toggle, then you will want to remove that so that the end-user is only prompted once.

Windows Runtime 8.x to UWP case study: Bookstore1

Article • 10/20/2022

This topic presents a case study of porting a very simple Universal 8.1 app to a Windows 10 Universal Windows Platform (UWP) app. A Universal 8.1 app is one that builds one app package for Windows 8.1, and a different app package for Windows Phone 8.1. With Windows 10, you can create a single app package that your customers can install onto a wide range of devices, and that's what we'll do in this case study. See Guide to UWP apps.

The app we'll port consists of a **ListBox** bound to a view model. The view model has a list of books that shows title, author, and book cover. The book cover images have **Build Action** set to **Content** and **Copy to Output Directory** set to **Do not copy**.

The previous topics in this section describe the differences between the platforms, and they give details and guidance on the porting process for various aspects of an app from XAML markup, through binding to a view model, down to accessing data. A case study aims to complement that guidance by showing it in action in a real example. The case studies assume you've read the guidance, which they do not repeat.

Note When opening Bookstore1Universal_10 in Visual Studio, if you see the message "Visual Studio update required", then follow the steps in TargetPlatformVersion.

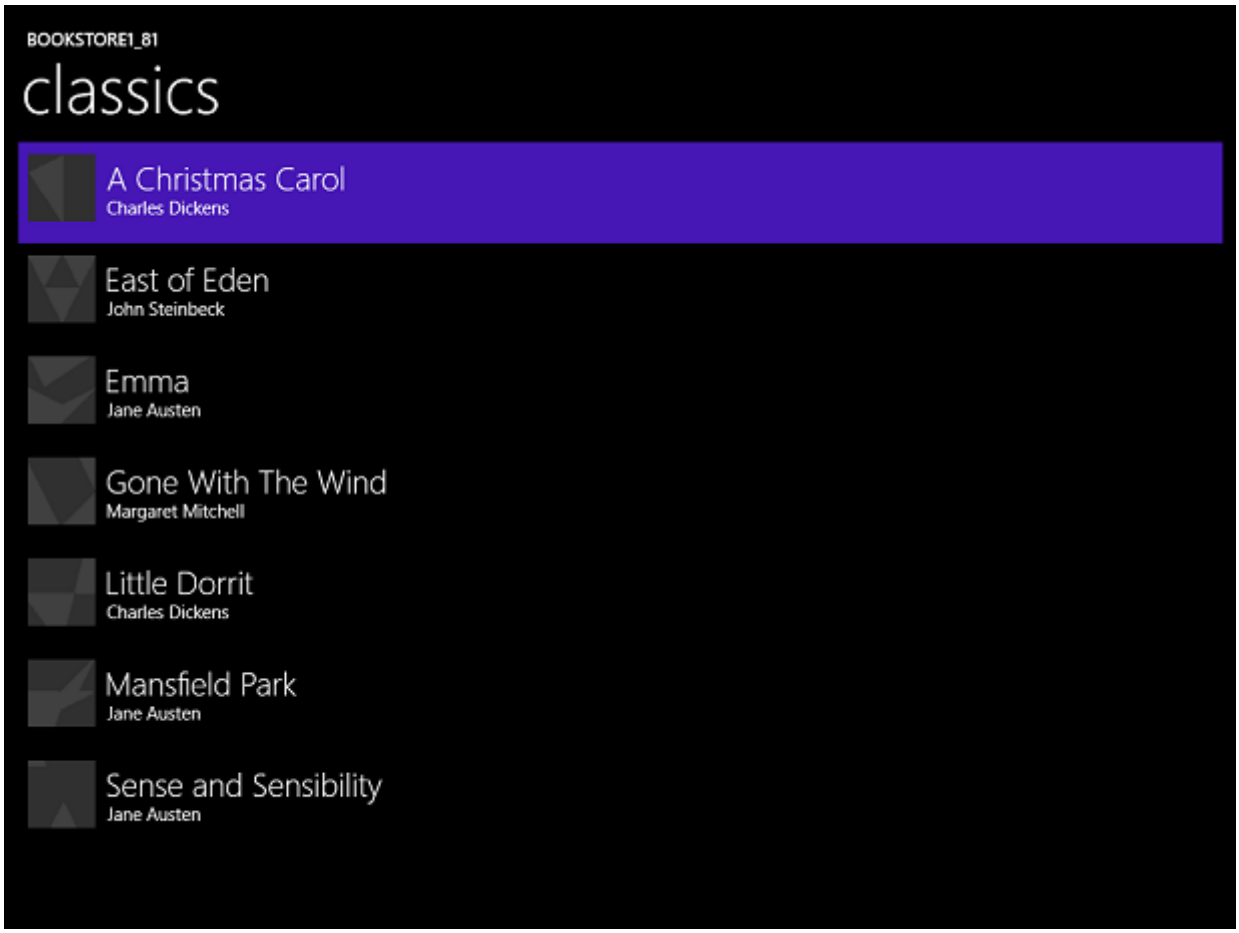
Downloads

Download the Bookstore1_81 Universal 8.1 app.

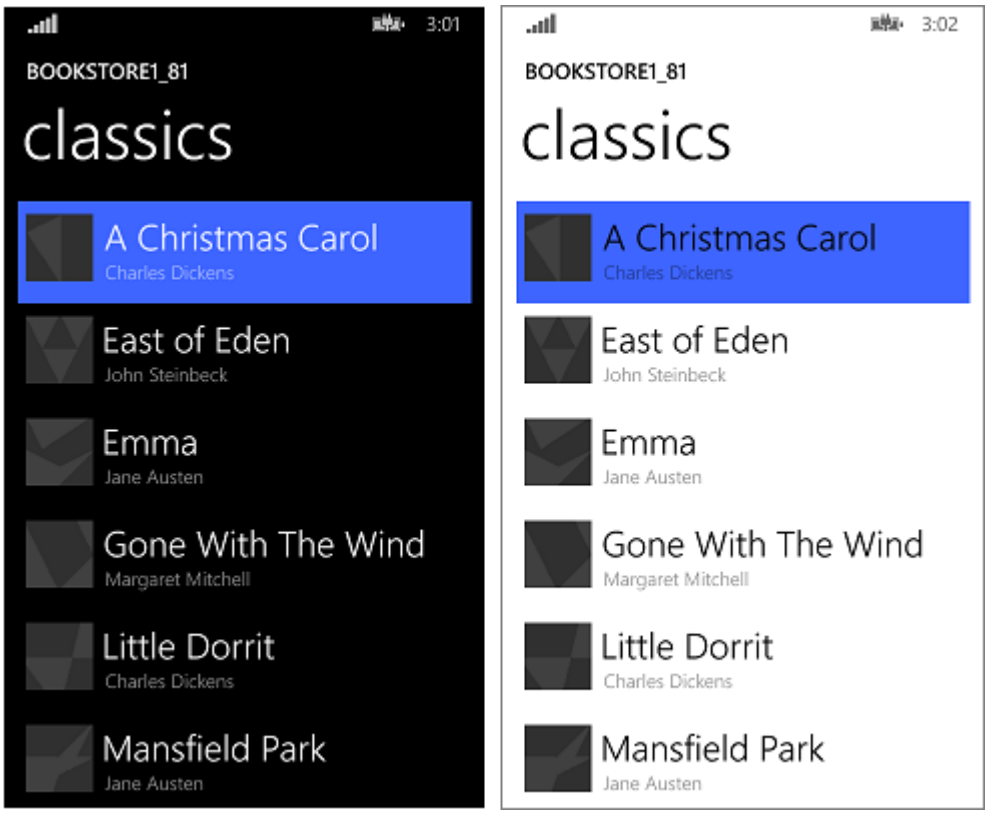
Download the Bookstore1Universal_10 Windows 10 app.

The Universal 8.1 app

Here's what Bookstore1_81—the app that we're going to port—looks like. It's just a vertically-scrolling list box of books beneath the heading of the app's name and page title.



Bookstore1_81 on Windows



Bookstore1_81 on Windows Phone

Porting to a Windows 10 project

The Bookstore1_81 solution is an 8.1 Universal App project, and it contains these projects.

- Bookstore1_81.Windows. This is the project that builds the app package for Windows 8.1.
- Bookstore1_81.WindowsPhone. This is the project that builds the app package for Windows Phone 8.1.
- Bookstore1_81.Shared. This is the project that contains source code, markup files, and other assets and resources, that are used by both of the other two projects.

For this case study, we have the usual options described in If you have a Universal 8.1 app with respect to what devices to support. The decision here is a simple one: this app has the same features, and does so mostly with the same code, in both its Windows 8.1 and Windows Phone 8.1 forms. So, we'll port the contents of the Shared project (and anything else we need from the other projects) to a Windows 10 that targets the Universal device family (one that you can install onto the widest range of devices).

It's a very quick task to create a new project in Visual Studio, copy files over to it from Bookstore1_81, and include the copied files in the new project. Start by creating a new Blank Application (Windows Universal) project. Name it Bookstore1Universal_10. These are the files to copy over from Bookstore1_81 to Bookstore1Universal_10.

From the Shared project

- Copy the folder containing the book cover image PNG files (the folder is \Assets\CoverImages). After copying the folder, in **Solution Explorer**, make sure **Show All Files** is toggled on. Right-click the folder that you copied and click **Include In Project**. That command is what we mean by "including" files or folders in a project. Each time you copy a file or folder, each copy, click **Refresh** in **Solution Explorer** and then include the file or folder in the project. There's no need to do this for files that you're replacing in the destination.
- Copy the folder containing the view model source file (the folder is \ViewModel).
- Copy MainPage.xaml and replace the file in the destination.

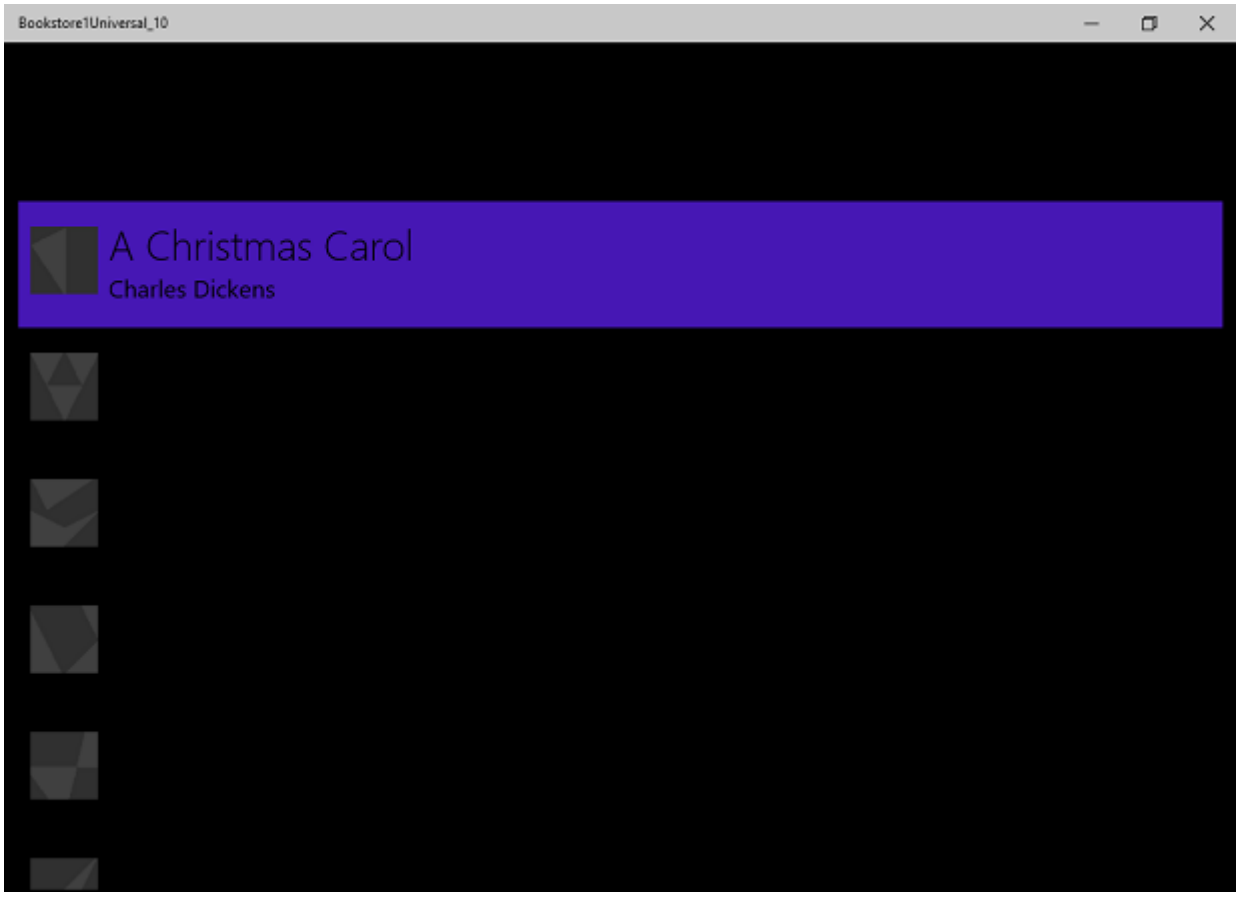
From the Windows project

- Copy BookstoreStyles.xaml. We'll use this one as a good starting-point because all the resource keys in this file will resolve in a Windows 10 app; some of those in the equivalent WindowsPhone file will not.

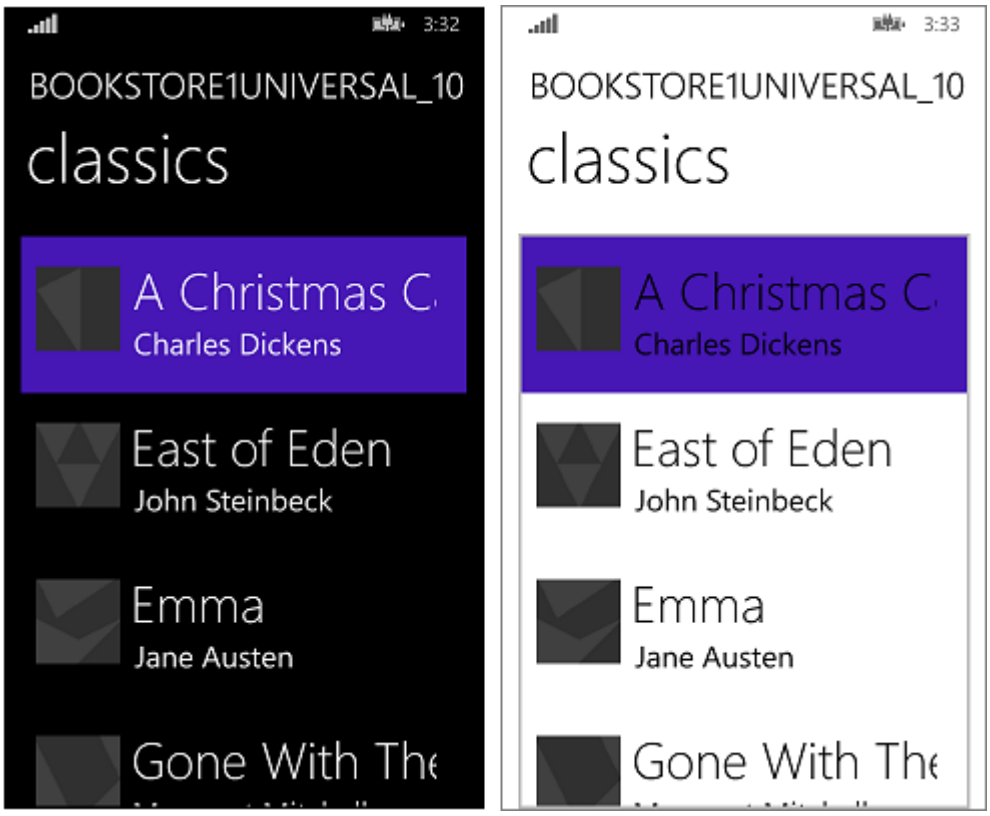
Edit the source code and markup files that you just copied and change any references to the Bookstore1_81 namespace to Bookstore1Universal_10. A quick way to do that is to use the **Replace In Files** feature. No code changes are needed in the view model, nor in

any other imperative code. But, just to make it easier to see which version of the app is running, change the value returned by the **Bookstore1Universal_10.BookstoreViewModel.AppName** property from "BOOKSTORE1_81" to "BOOKSTORE1UNIVERSAL_10".

Right now, you can build and run. Here's how our new UWP app looks after having done no explicit work yet to port it to Windows 10.



The Windows 10 app with initial source code changes running on a Desktop device



The Windows 10 app with initial source code changes running on a Mobile device

The view and the view model are working together correctly, and the **ListBox** is functioning. We just need to fix the styling. On a Mobile device, in light theme, we can see the border of the list box, but that will be easy to hide. And, the typography is too big, so we'll change the styles we're using. Also, the app should be light in color when running on a Desktop device if we want it to look like the default. So, we'll change that.

Universal styling

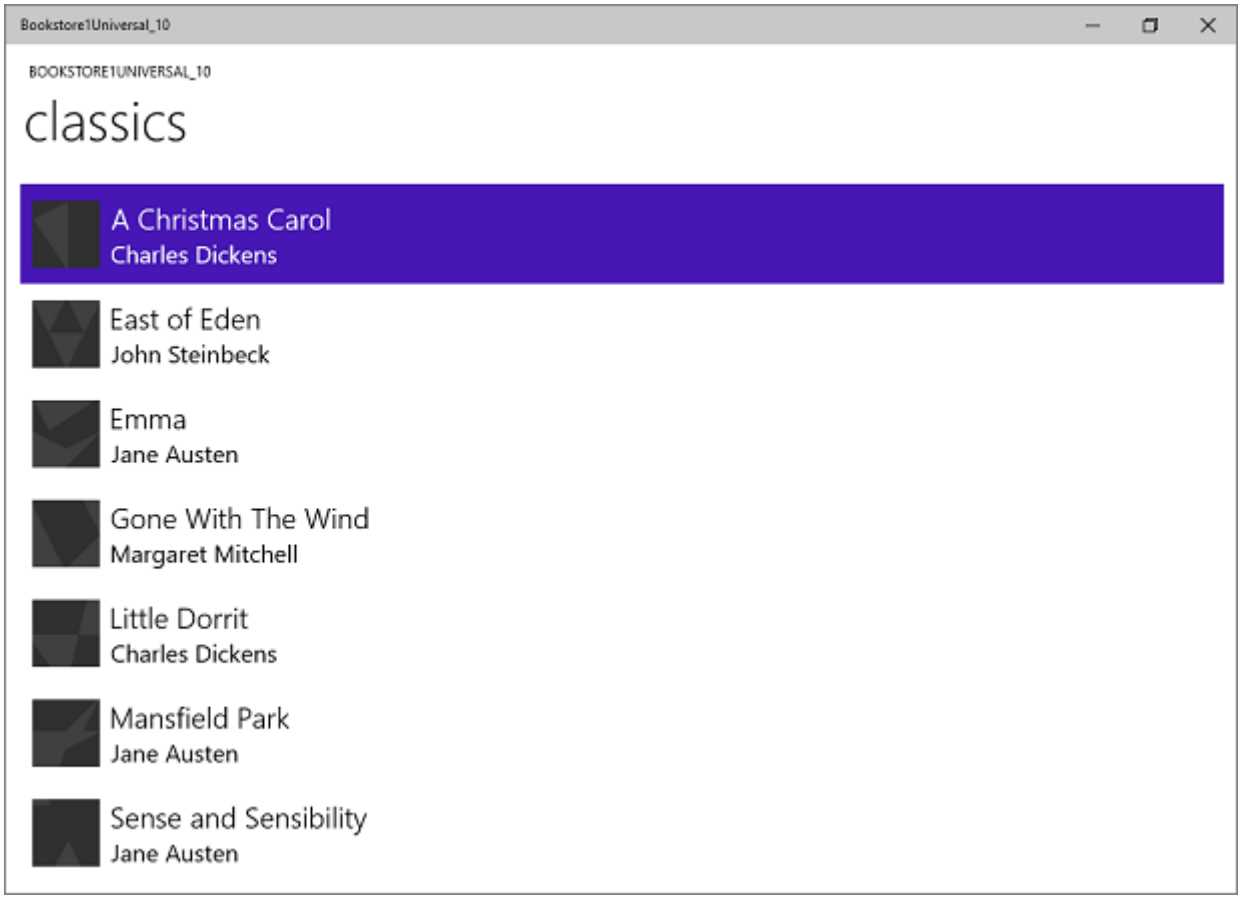
The Bookstore1_81 app used two different resource dictionaries (BookstoreStyles.xaml) to tailor its styles to the Windows 8.1 and Windows Phone 8.1 operating systems. Neither of those two BookstoreStyles.xaml files contains exactly the styles we need for our Windows 10 app. But, the good news is that what we want is actually much simpler than either of them. So, the next steps will mostly involve removing and simplifying our project files and markup. The steps are below. And you can use the links at the top of this topic to download the projects and see the results of all the changes between here and the end of the case study.

- To tighten up the spacing between items, find the `BookTemplate` data template in MainPage.xaml and delete the `Margin="0,0,0,8"` from the root **Grid**.
- Also, in `BookTemplate`, there are references to `BookTemplateTitleTextBlockStyle` and `BookTemplateAuthorTextBlockStyle`. Bookstore1_81 used those keys as an indirection so that a single key had different implementations in the two apps. We

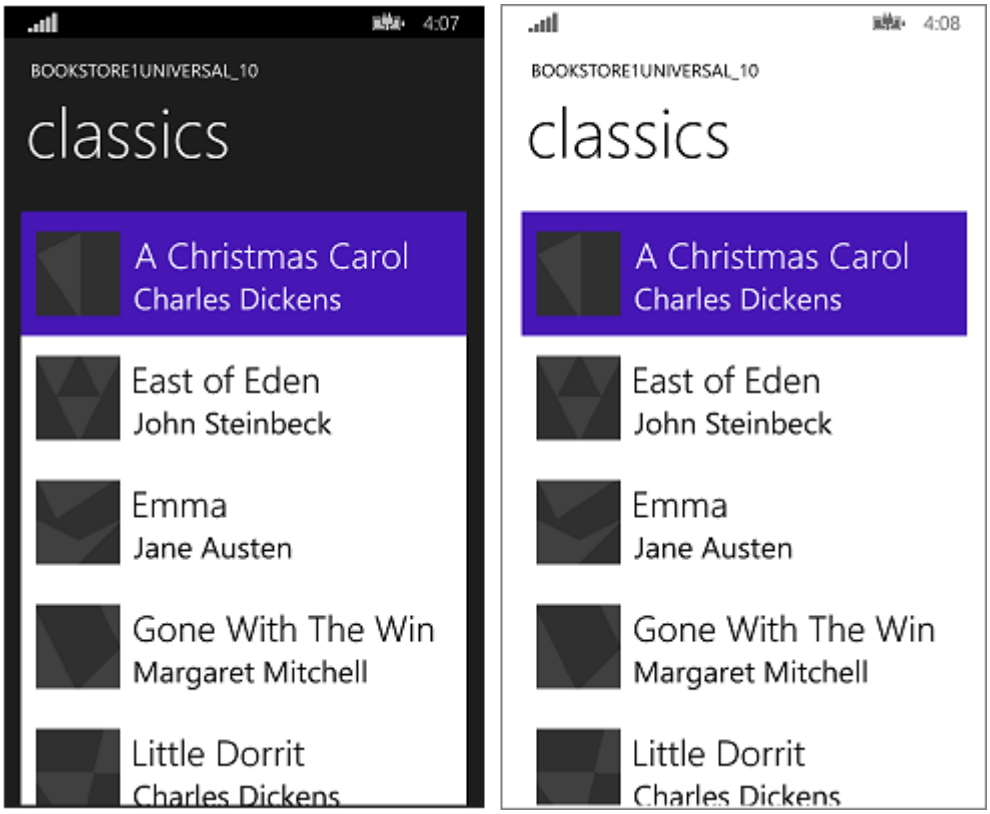
don't need that indirection any more; we can just reference system styles directly. So, replace those references with `TitleTextBlockStyle` and `SubtitleTextBlockStyle`, respectively.

- Now, we need to set `LayoutRoot`'s Background to the correct default value so that the app looks appropriate when running on all devices no matter what the theme is. Change it from `"Transparent"` to `"{ThemeResource ApplicationPageBackgroundThemeBrush}"`.
- In `TitlePanel`, change the reference to `TitleTextBlockStyle` (which is now a little too big) to a reference to `CaptionTextBlockStyle`. `PageTitleTextBlockStyle` is another Bookstore1_81 indirection that we don't need any longer. Change that to reference `HeaderTextBlockStyle` instead.
- We no longer need to set any special Background, Style, nor ItemContainerStyle on the **ListBox**, so just delete those three attributes and their values from the markup. We do want to hide the border of the **ListBox**, though, so add `BorderBrush="{x:Null}"` to it.
- We're not referencing any of the resources in the BookstoreStyles.xaml **ResourceDictionary** file any longer. You can delete all of those resources. But, don't delete the BookstoreStyles.xaml file itself: we still have one last use for it, as you'll see in the next section.

That last sequence of styling operations leaves the app looking like this.



The almost-ported Windows 10 app running on a Desktop device



The almost-ported Windows 10 app running on a Mobile device

An optional adjustment to the list box for Mobile devices

When the app is running on a Mobile device, the background of a list box is light by default in both themes. That may be the style that you prefer and, if so, then there's nothing more to do except to tidy up: delete the BookstoreStyles.xaml resource dictionary file from your project, and remove the markup that merges it into MainPage.xaml.

But, controls are designed so that you can customize their look while leaving their behavior unaffected. So, if you want the list box to be dark in the dark theme—the way the original app looked—then this section describes a way to do that.

The change we make only needs to affect the app when it's running on Mobile devices. So, we'll use a very slightly customized list box style when we're running on the Mobile device family, and we'll continue to use the default style when we're running everywhere else. To do that, we'll make a copy of BookstoreStyles.xaml and we'll give it a special MRT-qualified name, which will cause it to be loaded only on Mobile devices.

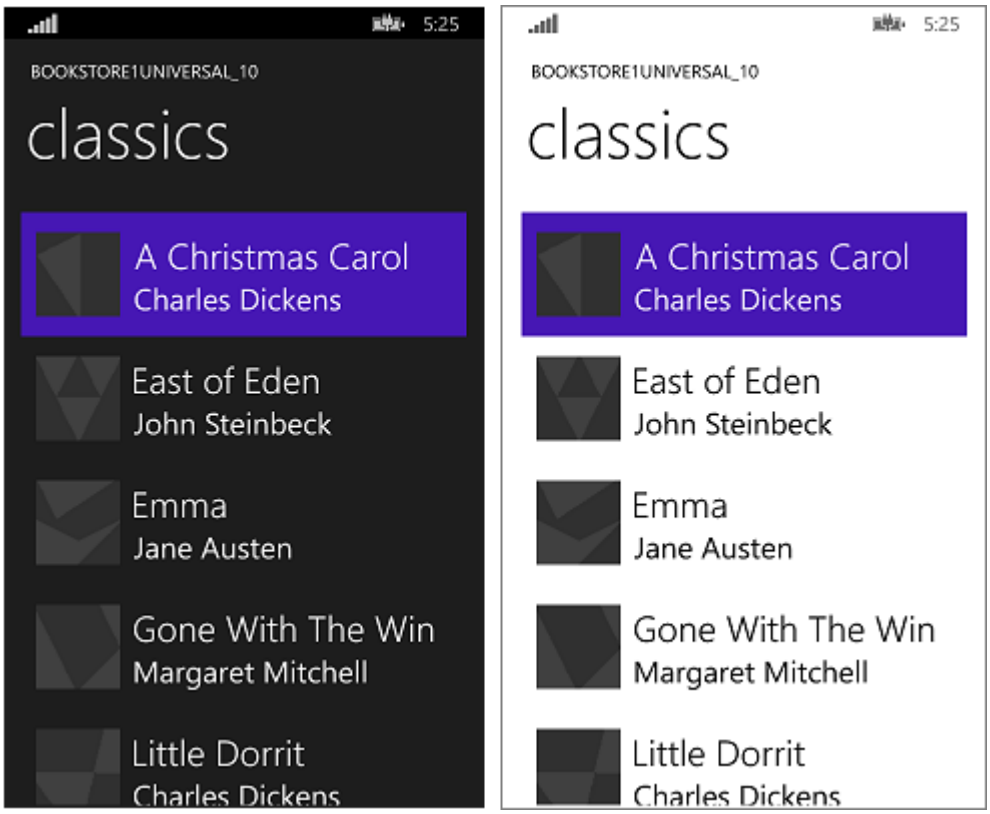
Add a new **ResourceDictionary** project item and name it BookstoreStyles.DeviceFamily-Mobile.xaml. You now have two files both of whose logical name is BookstoreStyles.xaml

(and that's the name you use in your markup and code). The files have different physical names, though, so they can contain different markup. You can use this MRT-qualified naming scheme with any xaml file, but be aware that all xaml files with the same logical name share a single xaml.cs code-behind file (where one is applicable).

Edit a copy of the control template for the list box and store that with the key of `BookstoreListBoxStyle` in the new resource dictionary, BookstoreStyles.DeviceFamily-Mobile.xaml. Now, we'll make simple changes to three of the setters.

- In the Foreground setter, change the value to `"{x:Null}"`. Note that setting a property to `"{x:Null}"` directly on an element is the same as setting it to `null` in code. But, using a value of `"{x:Null}"` in a setter has a unique effect: it overrides the setter in the default style (for the same property) and restores the default value of the property on the target element.
- In the Background setter, change the value to `"Transparent"` to remove that light background.
- In the Template setter, find the visual state named `Focused` and delete its Storyboard, making it into an empty tag.
- Delete all the other setters from the markup.

Finally, copy `BookstoreListBoxStyle` into BookstoreStyles.xaml and delete its three setters, making it into an empty tag. We do this so that on devices other than Mobile ones, our reference to BookstoreStyles.xaml and to `BookstoreListBoxStyle` will resolve, but will have no effect.



The ported Windows 10 app running on a Mobile device

Conclusion

This case study showed the process of porting a very simple app—arguably an unrealistically simple one. For instance, a list box can be used for selection or for establishing a context for navigation; the app navigates to a page with more details about the item that was tapped. This particular app does nothing with the user's selection, and it has no navigation. Even so, the case study served to break the ice, to introduce the porting process, and to demonstrate important techniques that you can use in real UWP apps.

We also saw evidence that porting view models is generally a smooth process. User interface, and form factor support, are aspects that are more likely to require our attention when porting.

The next case study is Bookstore2, in which we look at accessing and displaying grouped data.

Windows Runtime 8.x to UWP case study: Bookstore2

Article • 10/20/2022

This case study—which builds on the info given in Bookstore1—begins with a Universal 8.1 app that displays grouped data in a **SemanticZoom** control. In the view model, each instance of the class **Author** represents the group of the books written by that author, and in the **SemanticZoom**, we can either view the list of books grouped by author or we can zoom out to see a jump list of authors. The jump list affords much quicker navigation than scrolling through the list of books. We walk through the steps of porting the app to a Windows 10 Universal Windows Platform (UWP) app.

Note When opening Bookstore2Universal_10 in Visual Studio, if you see the message "Visual Studio update required", then follow the steps in TargetPlatformVersion.

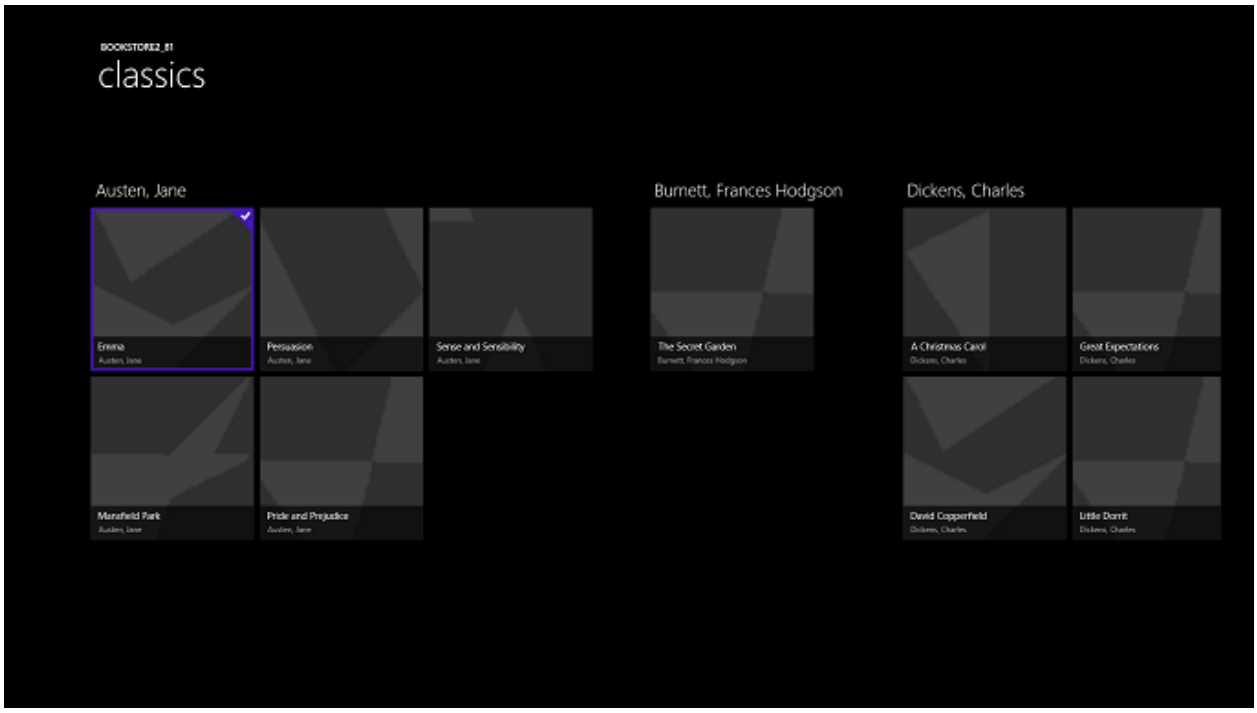
Downloads

Download the Bookstore2_81 Universal 8.1 app.

Download the Bookstore2Universal_10 Windows 10 app.

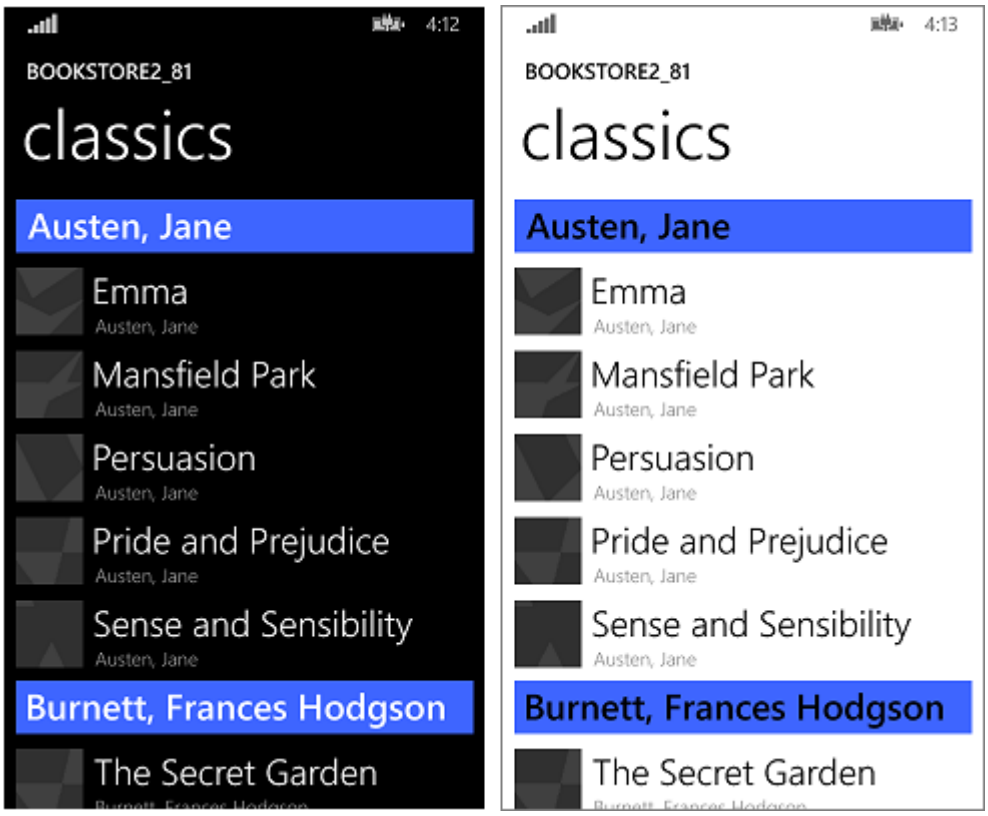
The Universal 8.1 app

Here's what Bookstore2_81—the app that we're going to port—looks like. It's a horizontally-scrolling (vertically-scrolling on Windows Phone) **SemanticZoom** showing books grouped by author. You can zoom out to the jump list and from there you can navigate back into any group. There are two main pieces to this app: the view model that provides the grouped data source, and the user interface that binds to that view model. As we'll see, both of these pieces port easily from WinRT 8.1 technology to Windows 10.



Bookstore2_81 on Windows, zoomed-in view

Bookstore2_81 on Windows, zoomed-out view



Bookstore2_81 on Windows Phone, zoomed-in view

Bookstore2_81 on Windows Phone, zoomed-out view

Porting to a Windows 10 project

The Bookstore2_81 solution is an 8.1 Universal App project. The Bookstore2_81.Windows project builds the app package for Windows 8.1, and the Bookstore2_81.WindowsPhone project builds the app package for Windows Phone 8.1. Bookstore2_81.Shared is the project that contains source code, markup files, and other assets and resources, that are used by both of the other two projects.

Just like with the previous case study, the option we'll take (of the ones described in If you have a Universal 8.1 app) is to port the contents of the Shared project to a Windows 10 that targets the Universal device family.

Begin by creating a new Blank Application (Windows Universal) project. Name it Bookstore2Universal_10. These are the files to copy over from Bookstore2_81 to Bookstore2Universal_10.

From the Shared project

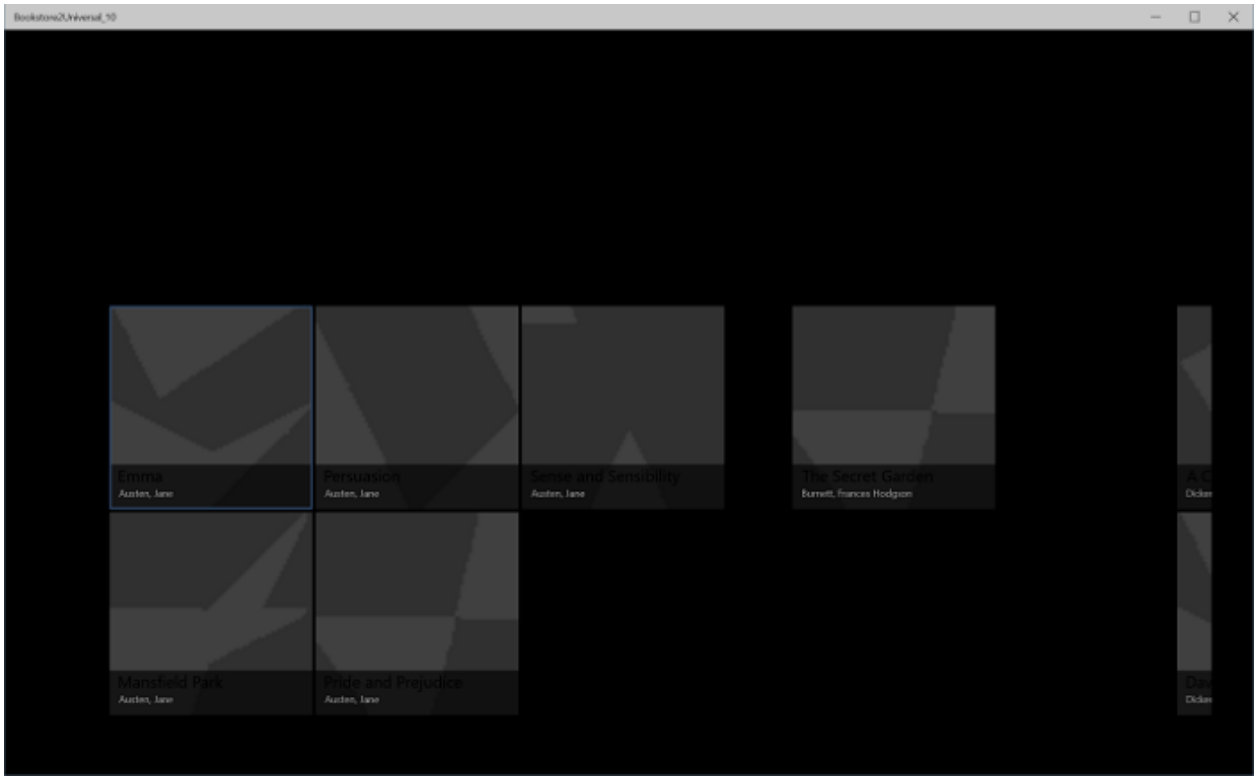
- Copy the folder containing the book cover image PNG files (the folder is \Assets\CoverImages). After copying the folder, in **Solution Explorer**, make sure **Show All Files** is toggled on. Right-click the folder that you copied and click **Include In Project**. That command is what we mean by "including" files or folders in a project. Each time you copy a file or folder, each copy, click **Refresh** in **Solution Explorer** and then include the file or folder in the project. There's no need to do this for files that you're replacing in the destination.
- Copy the folder containing the view model source file (the folder is \ViewModel).
- Copy MainPage.xaml and replace the file in the destination.

From the Windows project

- Copy BookstoreStyles.xaml. We'll use this one as a good starting-point because all the resource keys in this file will resolve in a Windows 10 app; some of those in the equivalent WindowsPhone file will not.
- Copy SeZoUC.xaml and SeZoUC.xaml.cs. We'll start with the Windows version of this view, which is appropriate for wide windows, and then later we'll make it adapt to smaller windows and, consequently, smaller devices.

Edit the source code and markup files that you just copied and change any references to the Bookstore2_81 namespace to Bookstore2Universal_10. A quick way to do that is to use the **Replace In Files** feature. No code changes are needed in the view model, nor in any other imperative code. But, just to make it easier to see which version of the app is running, change the value returned by the **Bookstore2Universal_10.BookstoreViewModel.AppName** property from "Bookstore2_81" to "BOOKSTORE2UNIVERSAL_10".

Right now, you can build and run. Here's how our new UWP app looks after having done no work yet to port it to Windows 10.



The Windows 10 app with initial source code changes running on a Desktop device, zoomed-in view

The Windows 10 app with initial source code changes running on a Desktop device, zoomed-out view

The view model and the zoomed-in and zoomed-out views are working together correctly, although there are issues that make that a little hard to see. One issue is that the **SemanticZoom** doesn't scroll. This is because, in Windows 10, the default style of a **GridView** causes it to be laid out vertically (and the Windows 10 design guidelines recommend that we use it that way in new and in ported apps). But, horizontal scrolling settings in the custom items panel template that we copied from the Bookstore2_81 project (which was designed for the 8.1 app) are in conflict with vertical scrolling settings in the Windows 10 default style that is being applied as a result of us having ported to a Windows 10 app. A second thing is that the app doesn't yet adapt its user-interface to give the best experience in different-sized windows and on small devices. And, thirdly, the correct styles and brushes are not yet being used, resulting in much of the text being invisible (including the group headers that you can click to zoom out). So, in the next three sections (SemanticZoom and GridView design changes, Adaptive UI, and Universal styling) we'll remedy those three issues.

SemanticZoom and GridView design changes

The design changes in Windows 10 to the **SemanticZoom** control are described in the section SemanticZoom changes. We have no work to do in this section in response to those changes.

The changes to **GridView** are described in the section GridView/ListView changes. We have some very minor adjustments to make to adapt to those changes, as described below.

- In SeZoUC.xaml, in `ZoomedInItemsPanelTemplate`, set `Orientation="Horizontal"` and `GroupPadding="0,0,0,20"`.
- In SeZoUC.xaml, delete `ZoomedOutItemsPanelTemplate` and remove the `ItemsPanel` attribute from the zoomed-out view.

And that's it!

Adaptive UI

After that change, the UI layout that SeZoUC.xaml gives us is great for when the app is running in a wide window (which is only possible on a device with a large screen). When the app's window is narrow, though (which happens on a small device, and can also happen on a large device), the UI that we had in the Windows Phone Store app is arguably most appropriate.

We can use the adaptive Visual State Manager feature to achieve this. We'll set properties on visual elements so that, by default, the UI is laid out in the narrow state using the smaller templates that we were using in the Windows Phone Store app. Then, we'll detect when the app's window is wider-than-or-equal-to a specific size (measured in units of effective pixels), and in response, we'll change the properties of visual elements so that we get a larger, and wider, layout. We'll put those property changes in a visual state, and we'll use an adaptive trigger to continuously monitor and determine whether to apply that visual state, or not, depending on the width of the window in effective pixels. We're triggering on window width in this case, but it's possible to trigger on window height, too.

A minimum window width of 548 epx is appropriate for this use case because that's the size of the smallest device we would want to show the wide layout on. Phones are typically smaller than 548 epx so on a small device like that we'd remain in the default narrow layout. On a PC, the window will launch by default wide enough to trigger the switch to the wide state. From there, you'll be able to drag the window narrow enough to display two columns of the 250x250-sized items. A little narrower than that and the trigger will deactivate, the wide visual state will be removed, and the default narrow layout will be in effect.

So, what properties do we need to set—and change—to achieve these two different layouts? There are two alternatives and each entails a different approach.

1. We can put two **SemanticZoom** controls in our markup. One would be a copy of the markup that we were using in the Windows Runtime 8.x app (using **GridView** controls inside it), and collapsed by default. The other would be a copy of the markup that we were using in the Windows Phone Store app (using **ListView** controls inside it), and visible by default. The visual state would switch the visibility properties of the two **SemanticZoom** controls. This would require very little effort to achieve but this not, in general, a high-performance technique. So, if you use it, you should profile your app and make sure it is still meeting your performance goals.
2. We can use a single **SemanticZoom** containing **ListView** controls. To achieve our two layouts, in the wide visual state, we would change the properties of the **ListView** controls, including the templates that are applied to them, to cause them to lay out in the same way as a **GridView** does. This might perform better, but there are so many small differences between the various styles and templates of **GridView** and **ListView** and between their various item types that this is the more difficult solution to achieve. This solution is also tightly coupled to the way the default styles and templates are designed at this moment in time, giving us a solution that's fragile and sensitive to any future changes to the defaults.

In this case study, we're going to go with the first alternative. But, if you like, you can try the second one and see if that works better for you. Here are the steps to take to implement that first alternative.

- On the SemanticZoom in the markup in your new project, set `x:Name="wideSeZo"` and `Visibility="Collapsed"`.
- Go back to the Bookstore2_81.WindowsPhone project and open SeZoUC.xaml. Copy the SemanticZoom element markup out of that file and paste it immediately after `wideSeZo` in your new project. Set `x:Name="narrowSeZo"` on element that you just pasted.
- But `narrowSeZo` needs a couple of styles that we haven't copied yet. Again in Bookstore2_81.WindowsPhone, copy the two styles (`AuthorGroupHeaderContainerStyle` and `ZoomedOutAuthorItemContainerStyle`) out of SeZoUC.xaml and paste them into BookstoreStyles.xaml in your new project.
- You now have two SemanticZoom elements in your new SeZoUC.xaml. Wrap those two elements in a **Grid**.
- In BookstoreStyles.xaml in your new project, append the word `Wide` to these three resource keys (and to their references in SeZoUC.xaml, but only to the references inside `wideSeZo`): `AuthorGroupHeaderTemplate`, `ZoomedOutAuthorTemplate`, and `BookTemplate`.
- In the Bookstore2_81.WindowsPhone project, open BookstoreStyles.xaml. From this file, copy those same three resources (mentioned above), and the two jump list item converters, and the namespace prefix declaration Windows_UI_Xaml_Controls_Primitives, and paste them all into BookstoreStyles.xaml in your new project.
- Finally, in SeZoUC.xaml in your new project, add the appropriate Visual State Manager markup to the **Grid** that you added above.

XML

```
    <Grid>
        <VisualStateManager.VisualStateGroups>
            <VisualStateGroup>
                <VisualState x:Name="WideState">
                    <VisualState.StateTriggers>
                        <AdaptiveTrigger MinWindowWidth="548"/>
                    </VisualState.StateTriggers>
                    <VisualState.Setters>
                        <Setter Target="wideSeZo.Visibility"
Value="Visible"/>
                        <Setter Target="narrowSeZo.Visibility"
Value="Collapsed"/>
                    </VisualState.Setters>
                </VisualState>
            </VisualStateGroup>
```

```
        </VisualStateManager.VisualStateGroups>

    ...

    </Grid>
```

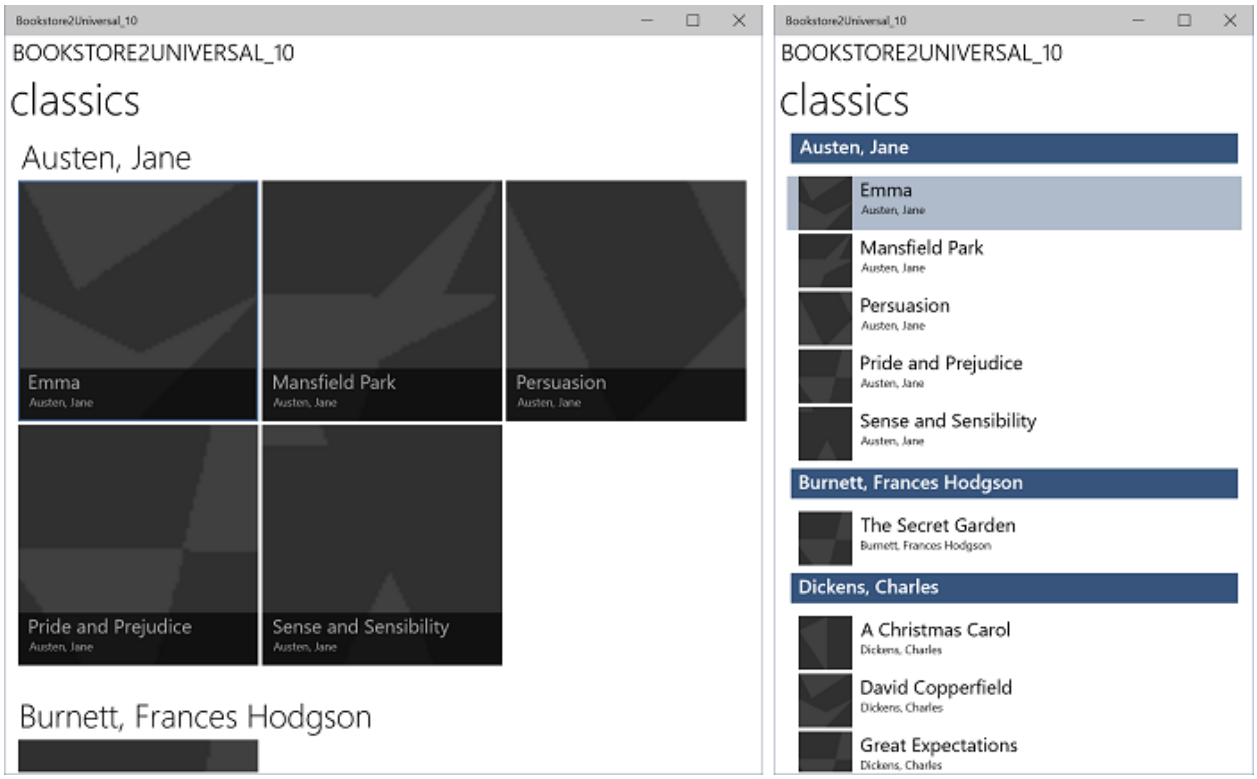
Universal styling

Now, let's fix up some styling issues, including one that we introduced above while copying from the old project.

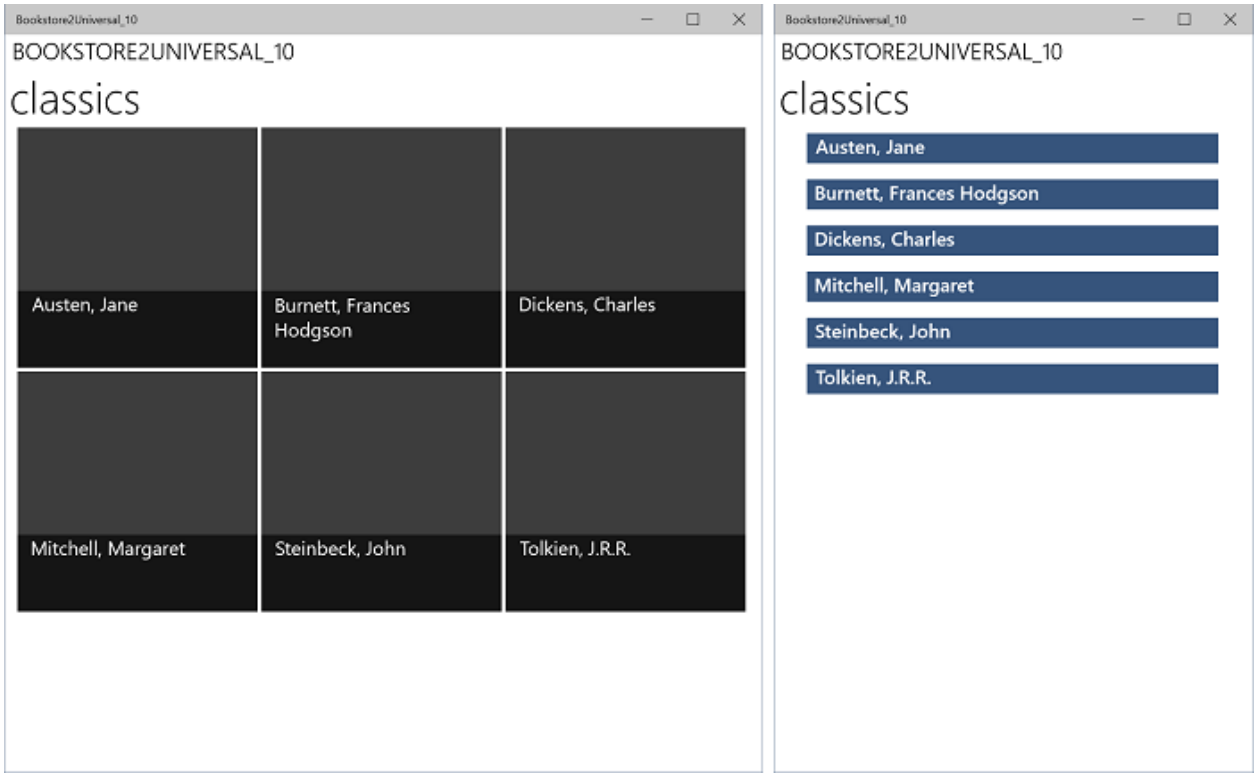
- In MainPage.xaml, change `LayoutRoot`'s Background to `"{ThemeResource ApplicationPageBackgroundThemeBrush}"`.
- In BookstoreStyles.xaml, set the value of the resource `TitlePanelMargin` to `0` (or whatever value looks good to you).
- In SeZoUC.xaml, set the Margin of `wideSeZo` to `0` (or whatever value looks good to you).
- In BookstoreStyles.xaml, remove the Margin attribute from `AuthorGroupHeaderTemplateWide`.
- Remove the FontFamily attribute from `AuthorGroupHeaderTemplate` and from `ZoomedOutAuthorTemplate`.
- Bookstore2_81 used the `BookTemplateTitleTextBlockStyle`, `BookTemplateAuthorTextBlockStyle`, and `PageTitleTextBlockStyle` resource keys as an indirection so that a single key had different implementations in the two apps. We don't need that indirection any more; we can just reference system styles directly. So, replace those references throughout the app with `TitleTextBlockStyle`, `CaptionTextBlockStyle`, and `HeaderTextBlockStyle` respectively. You can use the Visual Studio **Replace In Files** feature to do this quickly and accurately. You can then delete those three unused resources.
- In `AuthorGroupHeaderTemplate`, replace `PhoneAccentBrush` with `SystemControlBackgroundAccentBrush`, and set `Foreground="White"` on the **TextBlock** so that it looks correct when running on the mobile device family.
- In `BookTemplateWide`, copy the Foreground attribute from the second **TextBlock** to the first.
- In `ZoomedOutAuthorTemplateWide`, change the reference to `SubheaderTextBlockStyle` (which is now a little too big) to a reference to `SubtitleTextBlockStyle`.
- The zoomed-out view (the jump list) no longer overlays the zoomed-in view in the new platform, so we can remove the `Background` attribute from the zoomed-out view of `narrowSeZo`.

- So that all the styles and templates are in one file, move `ZoomedInItemsPanelTemplate` out of SeZoUC.xaml and into BookstoreStyles.xaml.

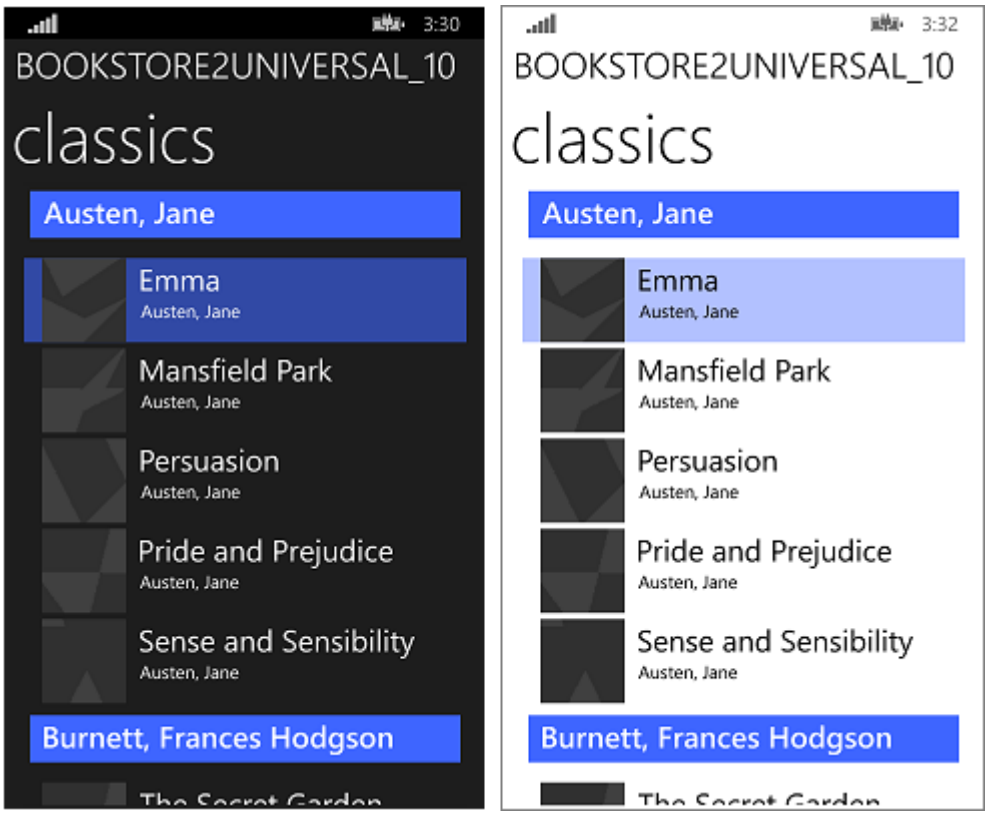
That last sequence of styling operations leaves the app looking like this.



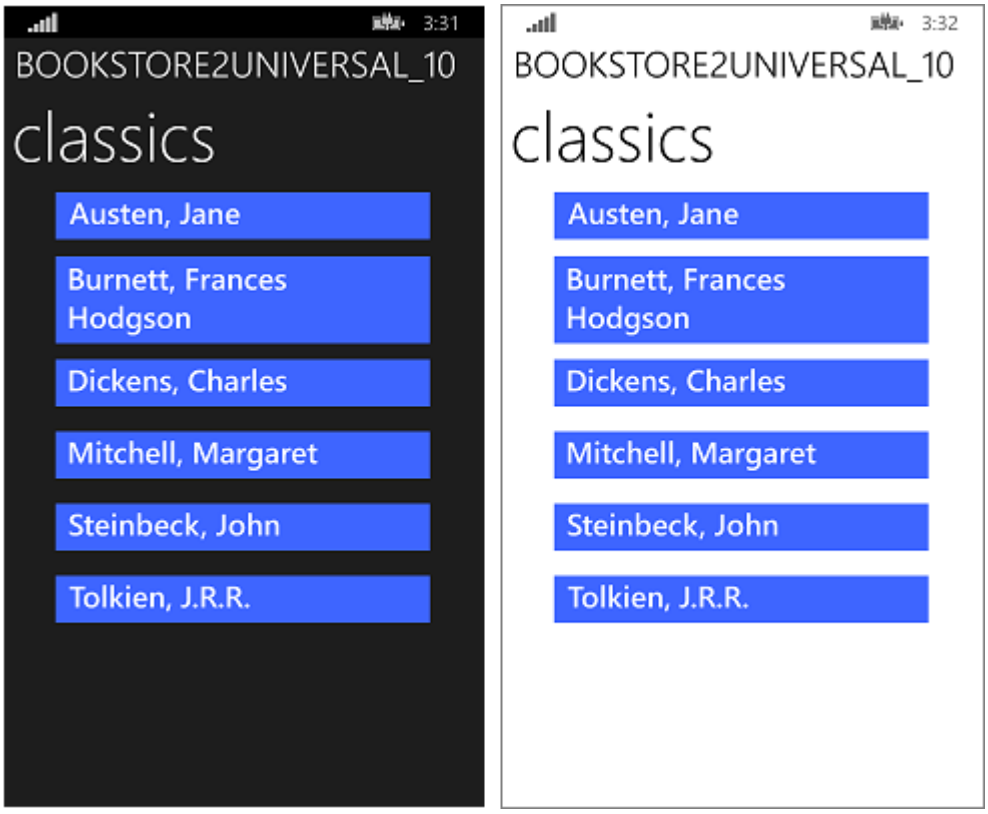
The ported Windows 10 app running on a Desktop device, zoomed-in view, two sizes of window



The ported Windows 10 app running on a Desktop device, zoomed-out view, two sizes of window



The ported Windows 10 app running on a Mobile device, zoomed-in view



The ported Windows 10 app running on a Mobile device, zoomed-out view

Conclusion

This case study involved a more ambitious user interface than the previous one. As with the previous case study, this particular view model required no work at all, and our efforts went primarily into refactoring the user interface. Some of the changes were a necessary result of combining two projects into one while still supporting many form factors (in fact, many more than we could before). A few of the changes were to do with changes that have been made to the platform.

The next case study is QuizGame, in which we look at accessing and displaying grouped data.

Windows Runtime 8.x to UWP case study: QuizGame sample app

Article • 10/20/2022

This topic presents a case study of porting a functioning peer-to-peer quiz game WinRT 8.1 sample app to a Windows 10 Universal Windows Platform (UWP) app.

A Universal 8.1 app is one that builds two versions of the same app: one app package for Windows 8.1, and a different app package for Windows Phone 8.1. The WinRT 8.1 version of QuizGame uses a Universal Windows app project arrangement, but it takes a different approach and it builds a functionally distinct app for the two platforms. The Windows 8.1 app package serves as the host for a quiz game session, while the Windows Phone 8.1 app package plays the role of the client to the host. The two halves of the quiz game session communicate via peer-to-peer networking.

Tailoring the two halves to PC, and phone, respectively makes good sense. But, wouldn't it be even better if you could run both the host and the client on just about any device of your choosing? In this case study, we'll port both apps to Windows 10 where they will each build into a single app package that users can install onto a wide range of devices.

The app uses patterns that make use of views and view models. As a result of this clean separation, the porting process for this app is very straightforward, as you'll see.

Note This sample assumes your network is configured to send and receive custom UDP group multicast packets (most home networks are, although your work network may not be). The sample also sends and receives TCP packets.

Note When opening QuizGame10 in Visual Studio, if you see the message "Visual Studio update required", then follow the steps in TargetPlatformVersion.

Downloads

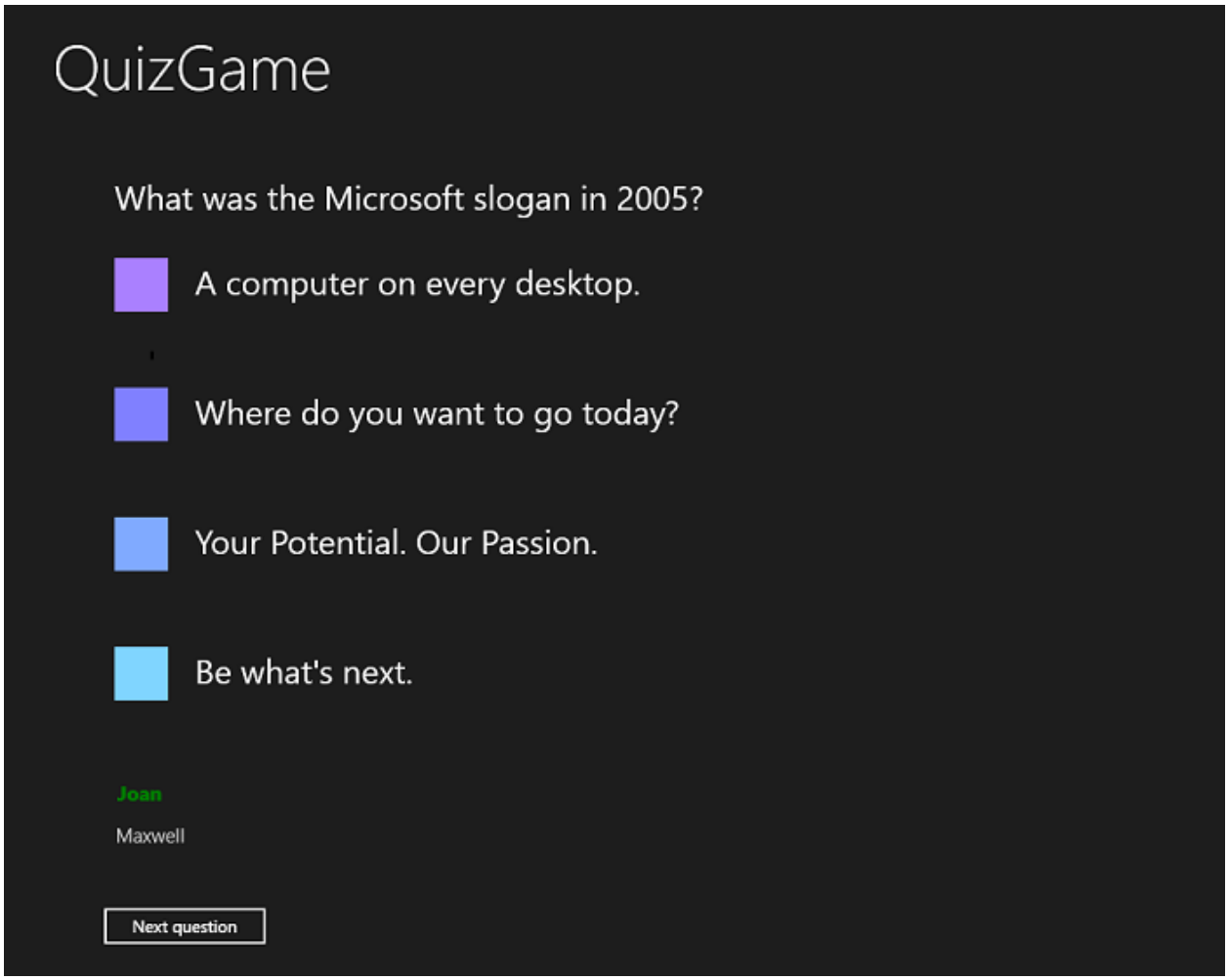
Download the QuizGame Universal 8.1 app. This is the initial state of the app prior to porting.

Download the QuizGame10 Windows 10 app. This is the state of the app just after porting.

See the latest version of this sample on GitHub.

The WinRT 8.1 solution

Here's what QuizGame—the app that we're going to port—looks like.



The QuizGame host app running on Windows